#S = immediate (I=1). S = register.
#D = immediate (L=1). D = register.

* Z = (result == 0).
** If #S and cogex, PC += signed(S). If #S and hubex, PC += signed(S*4). If S, PC = register S.

order	- Assembly Syntax -		- Group -	- Encoding -	- Alias -	- Description -	Next Inst Shielded from Interrupt	Clock Cycles (8 cogs) * +1 if crosses hub long - Cog Exec Mode - - LUT Exec Mode -	Clock Cycles (8 cogs) * +1 if crosses hub long - Hub Exec Mode -	Clock Cycles (16 cogs) * +1 if crosses hub long - Cog Exec Mode - - LUT Exec Mode -	Clock Cyles (16 cogs) * +1 if crosses hub long - Hub Exec Mode -	* Data not forwarded. - Register Write -	- Hub R/W -	- Stack R/W -
1	NOP		Miscellaneous	0000 0000000 000 000000000 000000000	.	No operation.		2	same	2	same			
2	ROR	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0000000 CZI DDDDDDDDD SSSSSSSSS	.	Rotate right. D = [31:0] of ({D[31:0], D[31:0]} >> S[4:0]). C = last bit shifted out if S[4:0] > 0, else D[0]. *		2	same	2	same	D		
3	ROL	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0000001 CZI DDDDDDDDD SSSSSSSSS	.	Rotate left. D = [63:32] of ({D[31:0], D[31:0]} << S[4:0]). C = last bit shifted out if S[4:0] > 0, else D[31]. *		2	same	2	same	D		
4	SHR	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0000010 CZI DDDDDDDDD SSSSSSSSS	.	Shift right. D = [31:0] of ({32'b0, D[31:0]} >> S[4:0]). C = last bit shifted out if S[4:0] > 0, else D[0]. *		2	same	2	same	D		
5	SHL	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0000011 CZI DDDDDDDDD SSSSSSSSS	.	Shift left. D = [63:32] of ({D[31:0], 32'b0} << S[4:0]). C = last bit shifted out if S[4:0] > 0, else D[31]. *		2	same	2	same	D		
6	RCR	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0000100 CZI DDDDDDDDD SSSSSSSSS	.	Rotate carry right. D = [31:0] of (((32{C}), D[31:0]) >> S[4:0]). C = last bit shifted out if S[4:0] > 0, else D[0]. *		2	same	2	same	D		
7	RCL	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0000101 CZI DDDDDDDDD SSSSSSSSS	.	Rotate carry left. D = [63:32] of ({D[31:0], (32{C})} << S[4:0]). C = last bit shifted out if S[4:0] > 0, else D[31]. *		2	same	2	same	D		
8	SAR	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0000110 CZI DDDDDDDDD SSSSSSSSS	.	Shift arithmetic right. D = [31:0] of (((32{D[31]}), D[31:0]) >> S[4:0]). C = last bit shifted out if S[4:0] > 0, else D[0]. *		2	same	2	same	D		
9	SAL	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0000111 CZI DDDDDDDDD SSSSSSSSS	.	Shift arithmetic left. D = [63:32] of ({D[31:0], (32{D[0]})} << S[4:0]). C = last bit shifted out if S[4:0] > 0, else D[31]. *		2	same	2	same	D		
10	ADD	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0001000 CZI DDDDDDDDD SSSSSSSSS	.	Add S into D. D = D + S. C = carry of (D + S). *		2	same	2	same	D		
11	ADDX	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0001001 CZI DDDDDDDDD SSSSSSSSS	.	Add (S + C) into D, extended. D = D + S + C. C = carry of (D + S + C). Z = Z AND (result == 0).		2	same	2	same	D		
12	ADDS	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0001010 CZI DDDDDDDDD SSSSSSSSS	.	Add S into D, signed. D = D + S. C = correct sign of (D + S). *		2	same	2	same	D		
13	ADDSX	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0001011 CZI DDDDDDDDD SSSSSSSSS	.	Add (S + C) into D, signed and extended. D = D + S + C. C = correct sign of (D + S + C). Z = Z AND (result == 0).		2	same	2	same	D		
14	SUB	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0001100 CZI DDDDDDDDD SSSSSSSSS	.	Subtract S from D. D = D - S. C = borrow of (D - S). *		2	same	2	same	D		
15	SUBX	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0001101 CZI DDDDDDDDD SSSSSSSSS	.	Subtract (S + C) from D, extended. D = D - (S + C). C = borrow of (D - (S + C)). Z = Z AND (result == 0).		2	same	2	same	D		
16	SUBS	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0001110 CZI DDDDDDDDD SSSSSSSSS	.	Subtract S from D, signed. D = D - S. C = correct sign of (D - S). *		2	same	2	same	D		
17	SUBSX	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0001111 CZI DDDDDDDDD SSSSSSSSS	.	Subtract (S + C) from D, signed and extended. D = D - (S + C). C = correct sign of (D - (S + C)). Z = Z AND (result == 0).		2	same	2	same	D		
18	CMP	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0010000 CZI DDDDDDDDD SSSSSSSSS	.	Compare D to S. C = borrow of (D - S). Z = (D == S).		2	same	2	same			
19	CMPX	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0010001 CZI DDDDDDDDD SSSSSSSSS	.	Compare D to (S + C), extended. C = borrow of (D - (S + C)). Z = Z AND (D == S + C).		2	same	2	same			
20	CMPS	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0010010 CZI DDDDDDDDD SSSSSSSSS	.	Compare D to S, signed. C = correct sign of (D - S). Z = (D == S).		2	same	2	same			
21	CMPSX	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0010011 CZI DDDDDDDDD SSSSSSSSS	.	Compare D to (S + C), signed and extended. C = correct sign of (D - (S + C)). Z = Z AND (D == S + C).		2	same	2	same			
22	CMPR	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0010100 CZI DDDDDDDDD SSSSSSSSS	.	Compare S to D (reverse). C = borrow of (S - D). Z = (D == S).		2	same	2	same			
23	CMPM	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0010101 CZI DDDDDDDDD SSSSSSSSS	.	Compare D to S, get MSB of difference into C. C = MSB of (D - S). Z = (D == S).		2	same	2	same			
24	SUBR	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0010110 CZI DDDDDDDDD SSSSSSSSS	.	Subtract D from S (reverse). D = S - D. C = borrow of (S - D). *		2	same	2	same	D		
25	CMPSUB	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0010111 CZI DDDDDDDDD SSSSSSSSS	.	Compare and subtract S from D if D >= S. If D => S then D = D - S and C = 1, else D same and C = 0. *		2	same	2	same	D		
26	FGE	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0011000 CZI DDDDDDDDD SSSSSSSSS	.	Force D >= S. If D < S then D = S and C = 1, else D same and C = 0. *		2	same	2	same	D		
27	FLE	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0011001 CZI DDDDDDDDD SSSSSSSSS	.	Force D <= S. If D > S then D = S and C = 1, else D same and C = 0. *		2	same	2	same	D		
28	FGES	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0011010 CZI DDDDDDDDD SSSSSSSSS	.	Force D >= S, signed. If D < S then D = S and C = 1, else D same and C = 0. *		2	same	2	same	D		
29	FLES	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0011011 CZI DDDDDDDDD SSSSSSSSS	.	Force D <= S, signed. If D > S then D = S and C = 1, else D same and C = 0. *		2	same	2	same	D		
30	SUMC	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0011100 CZI DDDDDDDDD SSSSSSSSS	.	Sum +/-S into D by C. If C = 1 then D = D - S, else D = D + S. C = correct sign of (D +/- S). *		2	same	2	same	D		
31	SUMNC	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0011101 CZI DDDDDDDDD SSSSSSSSS	.	Sum +/-S into D by !C. If C = 0 then D = D - S, else D = D + S. C = correct sign of (D +/- S). *		2	same	2	same	D		
32	SUMZ	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0011110 CZI DDDDDDDDD SSSSSSSSS	.	Sum +/-S into D by Z. If Z = 1 then D = D - S, else D = D + S. C = correct sign of (D +/- S). *		2	same	2	same	D		
33	SUMNZ	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0011111 CZI DDDDDDDDD SSSSSSSSS	.	Sum +/-S into D by !Z. If Z = 0 then D = D - S, else D = D + S. C = correct sign of (D +/- S). *		2	same	2	same	D		
34	TESTB	D,{#}S WC/WZ	Math and Logic	EEEE 0100000 CZI DDDDDDDDD SSSSSSSSS	.	Test bit S[4:0] of D, write to C/Z. C/Z = D[S[4:0]].		2	same	2	same			
35	TESTBN	D,{#}S WC/WZ	Math and Logic	EEEE 0100001 CZI DDDDDDDDD SSSSSSSSS	.	Test bit S[4:0] of !D, write to C/Z. C/Z = !D[S[4:0]].		2	same	2	same			
36	TESTB	D,{#}S ANDC/ANDZ	Math and Logic	EEEE 0100010 CZI DDDDDDDDD SSSSSSSSS	.	Test bit S[4:0] of D, AND into C/Z. C/Z = C/Z AND D[S[4:0]].		2	same	2	same			
37	TESTBN	D,{#}S ANDC/ANDZ	Math and Logic	EEEE 0100011 CZI DDDDDDDDD SSSSSSSSS	.	Test bit S[4:0] of !D, AND into C/Z. C/Z = C/Z AND !D[S[4:0]].		2	same	2	same			
38	TESTB	D,{#}S ORC/ORZ	Math and Logic	EEEE 0100100 CZI DDDDDDDDD SSSSSSSSS	.	Test bit S[4:0] of D, OR into C/Z. C/Z = C/Z OR D[S[4:0]].		2	same	2	same			
39	TESTBN	D,{#}S ORC/ORZ	Math and Logic	EEEE 0100101 CZI DDDDDDDDD SSSSSSSSS	.	Test bit S[4:0] of !D, OR into C/Z. C/Z = C/Z OR !D[S[4:0]].		2	same	2	same			
40	TESTB	D,{#}S XORC/XORZ	Math and Logic	EEEE 0100110 CZI DDDDDDDDD SSSSSSSSS	.	Test bit S[4:0] of D, XOR into C/Z. C/Z = C/Z XOR D[S[4:0]].		2	same	2	same			
41	TESTBN	D,{#}S XORC/XORZ	Math and Logic	EEEE 0100111 CZI DDDDDDDDD SSSSSSSSS	.	Test bit S[4:0] of !D, XOR into C/Z. C/Z = C/Z XOR !D[S[4:0]].		2	same	2	same			
42	BITL	D,{#}S {WCZ}	Math and Logic	EEEE 0100000 CZI DDDDDDDDD SSSSSSSSS	.	Bits D[S[9:5]+S[4:0]:S[4:0]] = 0. Other bits unaffected. Prior SETQ overrides S[9:5]. C,Z = original D[S[4:0]].		2	same	2	same	D		
43	BITH	D,{#}S {WCZ}	Math and Logic	EEEE 0100001 CZI DDDDDDDDD SSSSSSSSS	.	Bits D[S[9:5]+S[4:0]:S[4:0]] = 1. Other bits unaffected. Prior SETQ overrides S[9:5]. C,Z = original D[S[4:0]].		2	same	2	same	D		
44	BITC	D,{#}S {WCZ}	Math and Logic	EEEE 0100010 CZI DDDDDDDDD SSSSSSSSS	.	Bits D[S[9:5]+S[4:0]:S[4:0]] = C. Other bits unaffected. Prior SETQ overrides S[9:5]. C,Z = original D[S[4:0]].		2	same	2	same	D		
45	BITNC	D,{#}S {WCZ}	Math and Logic	EEEE 0100011 CZI DDDDDDDDD SSSSSSSSS	.	Bits D[S[9:5]+S[4:0]:S[4:0]] = !C. Other bits unaffected. Prior SETQ overrides S[9:5]. C,Z = original D[S[4:0]].		2	same	2	same	D		
46	BITZ	D,{#}S {WCZ}	Math and Logic	EEEE 0100100 CZI DDDDDDDDD SSSSSSSSS	.	Bits D[S[9:5]+S[4:0]:S[4:0]] = Z. Other bits unaffected. Prior SETQ overrides S[9:5]. C,Z = original D[S[4:0]].		2	same	2	same	D		
47	BITNZ	D,{#}S {WCZ}	Math and Logic	EEEE 0100101 CZI DDDDDDDDD SSSSSSSSS	.	Bits D[S[9:5]+S[4:0]:S[4:0]] = !Z. Other bits unaffected. Prior SETQ overrides S[9:5]. C,Z = original D[S[4:0]].		2	same	2	same	D		
48	BITRND	D,{#}S {WCZ}	Math and Logic	EEEE 0100110 CZI DDDDDDDDD SSSSSSSSS	.	Bits D[S[9:5]+S[4:0]:S[4:0]] = RNDs. Other bits unaffected. Prior SETQ overrides S[9:5]. C,Z = original D[S[4:0]].		2	same	2	same	D		
49	BITNOT	D,{#}S {WCZ}	Math and Logic	EEEE 0100111 CZI DDDDDDDDD SSSSSSSSS	.	Toggle bits D[S[9:5]+S[4:0]:S[4:0]]. Other bits unaffected. Prior SETQ overrides S[9:5]. C,Z = original D[S[4:0]].		2	same	2	same	D		
50	AND	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0101000 CZI DDDDDDDDD SSSSSSSSS	.	AND S into D. D = D & S. C = parity of result. *		2	same	2	same	D		
51	ANDN	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0101001 CZI DDDDDDDDD SSSSSSSSS	.	AND !S into D. D = D & !S. C = parity of result. *		2	same	2	same	D		
52	OR	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0101010 CZI DDDDDDDDD SSSSSSSSS	.	OR S into D. D = D | S. C = parity of result. *		2	same	2	same	D		
53	XOR	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0101011 CZI DDDDDDDDD SSSSSSSSS	.	XOR S into D. D = D ^ S. C = parity of result. *		2	same	2	same	D		
54	MUXC	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0101100 CZI DDDDDDDDD SSSSSSSSS	.	Mux C into each D bit that is '1' in S. D = (!S & D) | (S & (32{ C})). C = parity of result. *		2	same	2	same	D		
55	MUXNC	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0101101 CZI DDDDDDDDD SSSSSSSSS	.	Mux !C into each D bit that is '1' in S. D = (!S & D) | (S & (32{!C})). C = parity of result. *		2	same	2	same	D		
56	MUXZ	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0101110 CZI DDDDDDDDD SSSSSSSSS	.	Mux Z into each D bit that is '1' in S. D = (!S & D) | (S & (32{ Z})). C = parity of result. *		2	same	2	same	D		
57	MUXNZ	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0101111 CZI DDDDDDDDD SSSSSSSSS	.	Mux !Z into each D bit that is '1' in S. D = (!S & D) | (S & (32{!Z})). C = parity of result. *		2	same	2	same	D		
58	MOV	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0110000 CZI DDDDDDDDD SSSSSSSSS	.	Move S into D. D = S. C = S[31]. *		2	same	2	same	D		
59	NOT	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0110001 CZI DDDDDDDDD SSSSSSSSS	.	Get !S into D. D = !S. C = !S[31]. *		2	same	2	same	D		
60	NOT	D {WC/WZ/WCZ}	Math and Logic	EEEE 0110001 CZ0 DDDDDDDDD DDDDDDDDD	alias	Get !D into D. D = !D. C = !D[31]. *		2	same	2	same	D		
61	ABS	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0110010 CZI DDDDDDDDD SSSSSSSSS	.	Get absolute value of S into D. D = ABS(S). C = S[31]. *		2	same	2	same	D		
62	ABS	D {WC/WZ/WCZ}	Math and Logic	EEEE 0110010 CZ0 DDDDDDDDD DDDDDDDDD	alias	Get absolute value of D into D. D = ABS(D). C = D[31]. *		2	same	2	same	D		
63	NEG	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0110011 CZI DDDDDDDDD SSSSSSSSS	.	Negate S into D. D = -S. C = MSB of result. *		2	same	2	same	D		
64	NEG	D {WC/WZ/WCZ}	Math and Logic	EEEE 0110011 CZ0 DDDDDDDDD DDDDDDDDD	alias	Negate D. D = -D. C = MSB of result. *		2	same	2	same	D		
65	NEGC	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0110100 CZI DDDDDDDDD SSSSSSSSS	.	Negate S by C into D. If C = 1 then D = -S, else D = S. C = MSB of result. *		2	same	2	same	D		
66	NEGC	D {WC/WZ/WCZ}	Math and Logic	EEEE 0110100 CZ0 DDDDDDDDD DDDDDDDDD	alias	Negate D by C. If C = 1 then D = -D, else D = D. C = MSB of result. *		2	same	2	same	D		
67	NEGNC	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0110101 CZI DDDDDDDDD SSSSSSSSS	.	Negate S by !C into D. If C = 0 then D = -S, else D = S. C = MSB of result. *		2	same	2	same	D		
68	NEGNC	D {WC/WZ/WCZ}	Math and Logic	EEEE 0110101 CZ0 DDDDDDDDD DDDDDDDDD	alias	Negate D by !C. If C = 0 then D = -D, else D = D. C = MSB of result. *		2	same	2	same	D		
69	NEGZ	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0110110 CZI DDDDDDDDD SSSSSSSSS	.	Negate S by Z into D. If Z = 1 then D = -S, else D = S. C = MSB of result. *		2	same	2	same	D		
70	NEGZ	D {WC/WZ/WCZ}	Math and Logic	EEEE 0110110 CZ0 DDDDDDDDD DDDDDDDDD	alias	Negate D by Z. If Z = 1 then D = -D, else D = D. C = MSB of result. *		2	same	2	same	D		
71	NEGNZ	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0110111 CZI DDDDDDDDD SSSSSSSSS	.	Negate S by !Z into D. If Z = 0 then D = -S, else D = S. C = MSB of result. *		2	same	2	same	D		
72	NEGNZ	D {WC/WZ/WCZ}	Math and Logic	EEEE 0110111 CZ0 DDDDDDDDD DDDDDDDDD	alias	Negate D by !Z. If Z = 0 then D = -D, else D = D. C = MSB of result. *		2	same	2	same	D		
73	INCMOD	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0111000 CZI DDDDDDDDD SSSSSSSSS	.	Increment with modulus. If D = S then D = 0 and C = 1, else D = D + 1 and C = 0. *		2	same	2	same	D		
74	DECMOD	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0111001 CZI DDDDDDDDD SSSSSSSSS	.	Decrement with modulus. If D = 0 then D = S and C = 1, else D = D - 1 and C = 0. *		2	same	2	same	D		
75	ZEROX	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0111010 CZI DDDDDDDDD SSSSSSSSS	.	Zero-extend D above bit S[4:0]. C = MSB of result. *		2	same	2	same	D		
76	SIGNX	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0111011 CZI DDDDDDDDD SSSSSSSSS	.	Sign-extend D from bit S[4:0]. C = MSB of result. *		2	same	2	same	D		
77	ENCOD	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0111100 CZI DDDDDDDDD SSSSSSSSS	.	Get bit position of top-most '1' in S into D. D = position of top '1' in S (0..31). C = (S != 0). *		2	same	2	same	D		
78	ENCOD	D {WC/WZ/WCZ}	Math and Logic	EEEE 0111100 CZ0 DDDDDDDDD DDDDDDDDD	alias	Get bit position of top-most '1' in D into D. D = position of top '1' in S (0..31). C = (S != 0). *		2	same	2	same	D		
79	ONES	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0111101 CZI DDDDDDDDD SSSSSSSSS	.	Get number of '1's in S into D. D = number of '1's in S (0..32). C = LSB of result. *		2	same	2	same	D		
80	ONES	D {WC/WZ/WCZ}	Math and Logic	EEEE 0111101 CZ0 DDDDDDDDD DDDDDDDDD	alias	Get number of '1's in D into D. D = number of '1's in S (0..32). C = LSB of result. *		2	same	2	same	D		
81	TEST	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0111110 CZI DDDDDDDDD SSSSSSSSS	.	Test D with S. C = parity of (D & S). Z = ({D & S} == 0).		2	same	2	same			
82	TEST	D {WC/WZ/WCZ}	Math and Logic	EEEE 0111110 CZ0 DDDDDDDDD DDDDDDDDD	alias	Test D. C = parity of D. Z = (D == 0).		2	same	2	same			
83	TESTN	D,{#}S {WC/WZ/WCZ}	Math and Logic	EEEE 0111111 CZI DDDDDDDDD SSSSSSSSS	.	Test D with !S. C = parity of (D & !S). Z = ({D & !S} == 0).		2	same	2	same			
84	SETNIB	D,{#}S,#N	Math and Logic	EEEE 1000000 NNI DDDDDDDDD SSSSSSSSS	.	Set S[3:0] into nibble N in D, keeping rest of D same.		2	same	2	same	D		
85	SETNIB	{#}S	Math and Logic	EEEE 1000000 001 000000000 SSSSSSSSS	alias	Set S[3:0] into nibble established by prior ALTSN instruction.		2	same	2	same	D		
86	GETNIB	D,{#}S,#N	Math and Logic	EEEE 1000001 NNI DDDDDDDDD SSSSSSSSS	.	Get nibble N of S into D. D = {28'b0, S.NIBBLE[N]}.		2	same	2	same	D		
87	GETNIB	D	Math and Logic	EEEE 1000010 000 DDDDDDDDD 000000000	alias	Get nibble established by prior ALTGN instruction into D.		2	same	2	same	D		
88	ROLNIB	D,{#}S,#N	Math and Logic	EEEE 1000100 NNI DDDDDDDDD SSSSSSSSS	.	Rotate-left nibble N of S into D. D = {D[27:0], S.NIBBLE[N]}.		2	same	2	same	D		
89	ROLNIB	D	Math and Logic	EEEE 1000100 000 DDDDDDDDD 000000000	alias	Rotate-left nibble established by prior ALTGN instruction into D.		2	same	2	same	D		
90	SETBYTE	D,{#}S,#N	Math and Logic	EEEE 1000110 NNI DDDDDDDDD SSSSSSSSS	.	Set S[7:0] into byte N in D, keeping rest of D same.		2	same	2	same	D		
91	SETBYTE	{#}S	Math and Logic	EEEE 1000110 001 000000000 SSSSSSSSS	alias	Set S[7:0] into byte established by prior ALTSB instruction.		2	same	2	same	D		
92	GETBYTE	D,{#}S,#N	Math and Logic	EEEE 1000111 NNI DDDDDDDDD SSSSSSSSS	.	Get byte N of S into D. D = {24'b0, S.BYTE[N]}.		2	same	2	same	D		
93	GETBYTE	D	Math and Logic	EEEE 1000111 000 DDDDDDDDD 000000000	alias	Get byte established by prior ALTGB instruction into D.		2	same	2	same	D		
94	ROLBYTE	D,{#}S,#N	Math and Logic	EEEE 1001000 NNI DDDDDDDDD SSSSSSSSS	.	Rotate-left byte N of S into D. D = {D[23:0], S.BYTE[N]}.		2	same	2	same	D		
95	ROLBYTE	D	Math and Logic	EEEE 1001000 000 DDDDDDDDD 000000000	alias	Rotate-left byte established by prior ALTGB instruction into D.		2	same	2	same	D		
96	SETWORD	D,{#}S,#N	Math and Logic	EEEE 1001001 0NI DDDDDDDDD SSSSSSSSS	.	Set S[15:0] into word N in D, keeping rest of D same.		2	same	2	same	D		
97	SETWORD	{#}S	Math and Logic	EEEE 1001001 001 000000000 SSSSSSSSS	alias	Set S[15:0] into word established by prior ALTSW instruction.		2	same	2	same	D		
98	GETWORD	D,{#}S,#N	Math and Logic	EEEE 1001001 1NI DDDDDDDDD SSSSSSSSS	.	Get word N of S into D. D = {16'b0, S.WORD[N]}.		2	same	2	same	D		
99	GETWORD	D	Math and Logic	EEEE 1001001 100 DDDDDDDDD 000000000	alias	Get word established by prior ALTGW instruction into D.		2	same	2	same	D		
100	ROLWORD	D,{#}S,#N	Math and Logic	EEEE 1001010 0NI DDDDDDDDD SSSSSSSSS	.	Rotate-left word N of S into D. D = {D[15:0], S.WORD[N]}.		2	same	2	same	D		
101	ROLWORD	D	Math and Logic	EEEE 1001010 000 DDDDDDDDD 000000000	alias	Rotate-left word established by prior ALTGW instruction into D.		2	same	2	same	D		
102	ALTSN	D,{#}S	Register Indirection	EEEE 1001010 101 DDDDDDDDD SSSSSSSSS	.	Alter subsequent SETNIB instruction. Next D field = {D[11:3] + S) & $1FF, N field = D[2:0]. D += sign-extended S[17:9].	✔	2	same	2	same	D		
103	ALTSN	D	Register Indirection	EEEE 1001010 101 DDDDDDDDD 000000000	alias	Alter subsequent SETNIB instruction. Next D field = D[11:3], N field = D[2:0].	✔	2	same	2	same	D		
104	ALTGN	D,{#}S	Register Indirection	EEEE 1001010 111 DDDDDDDDD SSSSSSSSS	.	Alter subsequent GETNIB/ROLNIB instruction. Next S field = {D[11:3] + S) & $1FF, N field = D[2:0]. D += sign-extended S[17:9].	✔	2	same	2	same	D		
105	ALTGN	D	Register Indirection	EEEE 1001010 111 DDDDDDDDD 000000000	alias	Alter subsequent GETNIB/ROLNIB instruction. Next S field = D[11:3], N field = D[2:0].	✔	2	same	2	same	D		
106	ALTSB	D,{#}S	Register Indirection	EEEE 1001011 001 DDDDDDDDD SSSSSSSSS	.	Alter subsequent SETBYTE instruction. Next D field = {D[10:2] + S) & $1FF, N field = D[1:0]. D += sign-extended S[17:9].	✔	2	same	2	same	D		

#S = immediate (I=1). S = register.
#D = immediate (L=1). D = register.

* Z = (result == 0).
** If #S and cogex, PC += signed(S). If #S and hubex, PC += signed(S*4). If S, PC = register S.

* Data not forwarded.

order	- Assembly Syntax -	- Group -	- Encoding -	- Alias -	- Description -	Next Inst Shielded from Interrupt	Clock Cycles (8 cogs) * +1 if crosses hub long - Cog Exec Mode - - LUT Exec Mode -	Clock Cycles (8 cogs) * +1 if crosses hub long - Hub Exec Mode -	Clock Cycles (16 cogs) * +1 if crosses hub long - Cog Exec Mode - - LUT Exec Mode -	Clock Cyles (16 cogs) * +1 if crosses hub long - Hub Exec Mode -	- Register Write -	- Hub R/W -	- Stack R/W -
107	ALTSB D	Register Indirection	EEEE 1001011 001 DDDDDDDDD 000000000	alias	Alter subsequent SETBYTE instruction. Next D field = D[10:2], N field = D[1:0].	✔	2	same	2	same	D		
108	ALTGB D,{#}S	Register Indirection	EEEE 1001011 01I DDDDDDDDD SSSSSSSSS	.	Alter subsequent GETBYTE/ROLBYTE instruction. Next S field = {D[10:2] + S} & $1FF, N field = D[1:0]. D += sign-extended S[17:9].	✔	2	same	2	same	D		
109	ALTGB D	Register Indirection	EEEE 1001011 011 DDDDDDDDD 000000000	alias	Alter subsequent GETBYTE/ROLBYTE instruction. Next S field = D[10:2], N field = D[1:0].	✔	2	same	2	same	D		
110	ALTSW D,{#}S	Register Indirection	EEEE 1001011 10I DDDDDDDDD SSSSSSSSS	.	Alter subsequent SETWORD instruction. Next D field = {D[9:1] + S} & $1FF, N field = D[0]. D += sign-extended S[17:9].	✔	2	same	2	same	D		
111	ALTSW D	Register Indirection	EEEE 1001011 101 DDDDDDDDD 000000000	alias	Alter subsequent SETWORD instruction. Next D field = D[9:1], N field = D[0].	✔	2	same	2	same	D		
112	ALTGW D,{#}S	Register Indirection	EEEE 1001011 11I DDDDDDDDD SSSSSSSSS	.	Alter subsequent GETWORD/ROLWORD instruction. Next S field = {{D[9:1] + S} & $1FF}, N field = D[0]. D += sign-extended S[17:9].	✔	2	same	2	same	D		
113	ALTGW D	Register Indirection	EEEE 1001011 111 DDDDDDDDD 000000000	alias	Alter subsequent GETWORD/ROLWORD instruction. Next S field = D[9:1], N field = D[0].	✔	2	same	2	same	D		
114	ALTR D,{#}S	Register Indirection	EEEE 1001100 00I DDDDDDDDD SSSSSSSSS	.	Alter result register address (normally D field) of next instruction to (D + S) & $1FF. D += sign-extended S[17:9].	✔	2	same	2	same	D		
115	ALTR D	Register Indirection	EEEE 1001100 001 DDDDDDDDD 000000000	alias	Alter result register address (normally D field) of next instruction to D[8:0].	✔	2	same	2	same	D		
116	ALTD D,{#}S	Register Indirection	EEEE 1001100 01I DDDDDDDDD SSSSSSSSS	.	Alter D field of next instruction to (D + S) & $1FF. D += sign-extended S[17:9].	✔	2	same	2	same	D		
117	ALTD D	Register Indirection	EEEE 1001100 011 DDDDDDDDD 000000000	alias	Alter D field of next instruction to D[8:0].	✔	2	same	2	same	D		
118	ALTS D,{#}S	Register Indirection	EEEE 1001100 10I DDDDDDDDD SSSSSSSSS	.	Alter S field of next instruction to (D + S) & $1FF. D += sign-extended S[17:9].	✔	2	same	2	same	D		
119	ALTS D	Register Indirection	EEEE 1001100 101 DDDDDDDDD 000000000	alias	Alter S field of next instruction to D[8:0].	✔	2	same	2	same	D		
120	ALTB D,{#}S	Register Indirection	EEEE 1001100 11I DDDDDDDDD SSSSSSSSS	.	Alter D field of next instruction to {D[13:5] + S} & $1FF. D += sign-extended S[17:9].	✔	2	same	2	same	D		
121	ALTB D	Register Indirection	EEEE 1001100 111 DDDDDDDDD 000000000	alias	Alter D field of next instruction to D[13:5].	✔	2	same	2	same	D		
122	ALTI D,{#}S	Register Indirection	EEEE 1001101 00I DDDDDDDDD SSSSSSSSS	.	Substitute next instruction's I/R/D/S fields with fields from D, per S. Modify D per S.	✔	2	same	2	same	D		
123	ALTI D	Register Indirection	EEEE 1001101 001 DDDDDDDDD 101100100	alias	Execute D in place of next instruction. D stays same.	✔	2	same	2	same	D		
124	SETR D,{#}S	Math and Logic	EEEE 1001101 01I DDDDDDDDD SSSSSSSSS	.	Set R field of D to S[8:0]. D = {D[31:28], S[8:0], D[18:0]}.		2	same	2	same	D		
125	SETD D,{#}S	Math and Logic	EEEE 1001101 10I DDDDDDDDD SSSSSSSSS	.	Set D field of D to S[8:0]. D = {D[31:18], S[8:0], D[8:0]}.		2	same	2	same	D		
126	SETS D,{#}S	Math and Logic	EEEE 1001101 11I DDDDDDDDD SSSSSSSSS	.	Set S field of D to S[8:0]. D = {D[31:9], S[8:0]}.		2	same	2	same	D		
127	DECOD D,{#}S	Math and Logic	EEEE 1001110 00I DDDDDDDDD SSSSSSSSS	.	Decode S[4:0] into D. D = 1 << S[4:0].		2	same	2	same	D		
128	DECOD D	Math and Logic	EEEE 1001110 000 DDDDDDDDD DDDDDDDDD	alias	Decode D[4:0] into D. D = 1 << D[4:0].		2	same	2	same	D		
129	BMASK D,{#}S	Math and Logic	EEEE 1001110 01I DDDDDDDDD SSSSSSSSS	.	Get LSB-justified bit mask of size (S[4:0] + 1) into D. D = ($0_0000_0002 << S[4:0]) - 1.		2	same	2	same	D		
130	BMASK D	Math and Logic	EEEE 1001110 010 DDDDDDDDD DDDDDDDDD	alias	Get LSB-justified bit mask of size (D[4:0] + 1) into D. D = ($0_0000_0002 << D[4:0]) - 1.		2	same	2	same	D		
131	CRCBIT D,{#}S	Math and Logic	EEEE 1001110 10I DDDDDDDDD SSSSSSSSS	.	Iterate CRC value in D using C and polynomial in S. If (C XOR D[0]) then D = (D >> 1) XOR S, else D = (D >> 1).		2	same	2	same	D		
132	CRCNIB D,{#}S	Math and Logic	EEEE 1001110 11I DDDDDDDDD SSSSSSSSS	.	Iterate CRC value in D using Q[31:28] and polynomial in S. Like CRCBIT x 4. Q = Q << 4. For long, use SETQ+'REP #1,#8'+CRCNIB.	✔	2	same	2	same	D		
133	MUXNITS D,{#}S	Math and Logic	EEEE 1001111 001 DDDDDDDDD SSSSSSSSS	.	For each non-zero bit pair in S, copy that bit pair into the corresponding D bits, else leave that D bit pair the same.		2	same	2	same	D		
134	MUXNIBS D,{#}S	Math and Logic	EEEE 1001111 01I DDDDDDDDD SSSSSSSSS	.	For each non-zero nibble in S, copy that nibble into the corresponding D nibble, else leave that D nibble the same.		2	same	2	same	D		
135	MUXQ D,{#}S	Math and Logic	EEEE 1001111 10I DDDDDDDDD SSSSSSSSS	.	Used after SETQ. For each '1' bit in Q, copy the corresponding bit in S into D. D = (D & !Q) | (S & Q).		2	same	2	same	D		
136	MOVBYTS D,{#}S	Math and Logic	EEEE 1001111 11I DDDDDDDDD SSSSSSSSS	.	Move bytes within D, per S. D = {D.BYTE[S[7:6]], D.BYTE[S[5:4]], D.BYTE[S[3:2]], D.BYTE[S[1:0]]}.		2	same	2	same	D		
137	MUL D,{#}S {WZ}	Math and Logic	EEEE 1010000 0ZI DDDDDDDDD SSSSSSSSS	.	D = unsigned (D[15:0] * S[15:0]). Z = (S == 0) | (D == 0).		2	same	2	same	D		
138	MULS D,{#}S {WZ}	Math and Logic	EEEE 1010000 1ZI DDDDDDDDD SSSSSSSSS	.	D = signed (D[15:0] * S[15:0]). Z = (S == 0) | (D == 0).		2	same	2	same	D		
139	SCA D,{#}S {WZ}	Math and Logic	EEEE 1010001 0ZI DDDDDDDDD SSSSSSSSS	.	Next instruction's S value = unsigned (D[15:0] * S[15:0]) >> 16. *	✔	2	same	2	same			
140	SCAS D,{#}S {WZ}	Math and Logic	EEEE 1010001 1ZI DDDDDDDDD SSSSSSSSS	.	Next instruction's S value = signed (D[15:0] * S[15:0]) >> 14. In this scheme, $4000 = 1.0 and $C000 = -1.0. *	✔	2	same	2	same			
141	ADDPIX D,{#}S	Pixel Mixer	EEEE 1010010 00I DDDDDDDDD SSSSSSSSS	.	Add bytes of S into bytes of D, with $FF saturation.		7	same	7	same	D		
142	MULPIX D,{#}S	Pixel Mixer	EEEE 1010010 01I DDDDDDDDD SSSSSSSSS	.	Multiply bytes of S into bytes of D, where $FF = 1.0 and $00 = 0.0.		7	same	7	same	D		
143	BLNPIX D,{#}S	Pixel Mixer	EEEE 1010010 10I DDDDDDDDD SSSSSSSSS	.	Alpha-blend bytes of S into bytes of D, using SETPIV value.		7	same	7	same	D		
144	MIXPIX D,{#}S	Pixel Mixer	EEEE 1010010 11I DDDDDDDDD SSSSSSSSS	.	Mix bytes of S into bytes of D, using SETPIX and SETPIV values.		7	same	7	same	D		
145	ADDCT1 D,{#}S	Events - Configuration	EEEE 1010011 00I DDDDDDDDD SSSSSSSSS	.	Set CT1 event to trigger on CT = D + S. Adds S into D.		2	same	2	same	D		
146	ADDCT2 D,{#}S	Events - Configuration	EEEE 1010011 01I DDDDDDDDD SSSSSSSSS	.	Set CT2 event to trigger on CT = D + S. Adds S into D.		2	same	2	same	D		
147	ADDCT3 D,{#}S	Events - Configuration	EEEE 1010011 10I DDDDDDDDD SSSSSSSSS	.	Set CT3 event to trigger on CT = D + S. Adds S into D.		2	same	2	same	D		
148	WMLONG D,{#}S/P	Hub RAM - Write	EEEE 1010011 11I DDDDDDDDD SSSSSSSSS	.	Write only non-$00 bytes in D[31:0] to hub address (#)S/PTRx. Prior SETQ/SETQ2 invokes cog/LUT block transfer.		3...10 *	3...20 *	3...18 *	3...38 *		Write	
149	RQPIN D,{#}S {WC}	Smart Pins	EEEE 1010100 C0I DDDDDDDDD SSSSSSSSS	.	Read smart pin S[5:0] result "Z" into D, don't acknowledge pin ("Q" in RQPIN means "quiet"). C = modal result.		2	same	2	same	D		
150	RDPIN D,{#}S {WC}	Smart Pins	EEEE 1010100 C1I DDDDDDDDD SSSSSSSSS	.	Read smart pin S[5:0] result "Z" into D, acknowledge pin. C = modal result.		2	same	2	same	D		
151	RDLUT D,{#}S/P {WC/WZ/WCZ}	Lookup Table	EEEE 1010101 C0I DDDDDDDDD SSSSSSSSS	.	Read data from LUT address (#)S/PTRx into D. C = MSB of data. *		3	same	3	same	D		
152	RDBYTE D,{#}S/P {WC/WZ/WCZ}	Hub RAM - Read	EEEE 1010110 C0I DDDDDDDDD SSSSSSSSS	.	Read zero-extended byte from hub address (#)S/PTRx into D. C = MSB of byte. *		9...16	9...26	9...24	9...44	D	Read	
153	RDWORD D,{#}S/P {WC/WZ/WCZ}	Hub RAM - Read	EEEE 1010111 C0I DDDDDDDDD SSSSSSSSS	.	Read zero-extended word from hub address (#)S/PTRx into D. C = MSB of word. *		9...16 *	9...26 *	9...24 *	9...44 *	D	Read	
154	RDLONG D,{#}S/P {WC/WZ/WCZ}	Hub RAM - Read	EEEE 1011000 C0I DDDDDDDDD SSSSSSSSS	.	Read long from hub address (#)S/PTRx into D. C = MSB of long. * Prior SETQ/SETQ2 invokes cog/LUT block transfer.		9...16 *	9...26 *	9...24 *	9...44 *	D	Read	
155	POPA D {WC/WZ/WCZ}	Hub RAM - Read	EEEE 1011000 C0I DDDDDDDDD 101011111	alias	Read long from hub address --PTRA into D. C = MSB of long. *		9...16 *	9...26 *	9...24 *	9...44 *	D	Read	
156	POPB D {WC/WZ/WCZ}	Hub RAM - Read	EEEE 1011000 C0I DDDDDDDDD 111011111	alias	Read long from hub address --PTRB into D. C = MSB of long. *		9...16 *	9...26 *	9...24 *	9...44 *	D	Read	
157	CALLD D,{#}S {WC/WZ/WCZ}	Branch S - Call	EEEE 1011001 C0I DDDDDDDDD SSSSSSSSS	.	Call to S** by writing {C, Z, 10'b0, PC[19:0]} to D. C = S[31], Z = S[30].		4	13...20	4	13...28	D		
158	RESI3	Branch S - Resume	EEEE 1011001 110 111110000 111110001	alias	Resume from INT3. (CALLD $1F0,$1F1 WCZ)		4	13...20	4	13...28	D		
159	RESI2	Branch S - Resume	EEEE 1011001 110 111110010 111110011	alias	Resume from INT2. (CALLD $1F2,$1F3 WCZ)		4	13...20	4	13...28	D		
160	RESI1	Branch S - Resume	EEEE 1011001 110 111110100 111110101	alias	Resume from INT1. (CALLD $1F4,$1F5 WCZ)		4	13...20	4	13...28	D		
161	RESI0	Branch S - Resume	EEEE 1011001 110 111111110 111111111	alias	Resume from INT0. (CALLD $1FE,$1FF WCZ)		4	13...20	4	13...28	D		
162	RETI3	Branch S - Return	EEEE 1011001 110 111111111 111110001	alias	Return from INT3. (CALLD $1FF,$1F1 WCZ)		4	13...20	4	13...28	D		
163	RETI2	Branch S - Return	EEEE 1011001 110 111111111 111110011	alias	Return from INT2. (CALLD $1FF,$1F3 WCZ)		4	13...20	4	13...28	D		
164	RETI1	Branch S - Return	EEEE 1011001 110 111111111 111110101	alias	Return from INT1. (CALLD $1FF,$1F5 WCZ)		4	13...20	4	13...28	D		
165	RETI0	Branch S - Return	EEEE 1011001 110 111111111 111111111	alias	Return from INT0. (CALLD $1FF,$1FF WCZ)		4	13...20	4	13...28	D		
166	CALLPA {#}D,{#}S	Branch S - Call	EEEE 1011010 0LI DDDDDDDDD SSSSSSSSS	.	Call to S** by pushing {C, Z, 10'b0, PC[19:0]} onto stack, copy D to PA.		4	13...20	4	13...28	PA		Push
167	CALLPB {#}D,{#}S	Branch S - Call	EEEE 1011010 1LI DDDDDDDDD SSSSSSSSS	.	Call to S** by pushing {C, Z, 10'b0, PC[19:0]} onto stack, copy D to PB.		4	13...20	4	13...28	PB		Push
168	DJZ D,{#}S	Branch S - Mod & Test	EEEE 1011011 00I DDDDDDDDD SSSSSSSSS	.	Decrement D and jump to S** if result is zero.		2 or 4	2 or 13...20	2 or 4	2 or 13...28	D		
169	DJNZ D,{#}S	Branch S - Mod & Test	EEEE 1011011 01I DDDDDDDDD SSSSSSSSS	.	Decrement D and jump to S** if result is not zero.		2 or 4	2 or 13...20	2 or 4	2 or 13...28	D		
170	DJF D,{#}S	Branch S - Mod & Test	EEEE 1011011 10I DDDDDDDDD SSSSSSSSS	.	Decrement D and jump to S** if result is $FFFF_FFFF.		2 or 4	2 or 13...20	2 or 4	2 or 13...28	D		
171	DJNF D,{#}S	Branch S - Mod & Test	EEEE 1011011 11I DDDDDDDDD SSSSSSSSS	.	Decrement D and jump to S** if result is not $FFFF_FFFF.		2 or 4	2 or 13...20	2 or 4	2 or 13...28	D		
172	IJZ D,{#}S	Branch S - Mod & Test	EEEE 1011100 00I DDDDDDDDD SSSSSSSSS	.	Increment D and jump to S** if result is zero.		2 or 4	2 or 13...20	2 or 4	2 or 13...28	D		
173	IJNZ D,{#}S	Branch S - Mod & Test	EEEE 1011100 01I DDDDDDDDD SSSSSSSSS	.	Increment D and jump to S** if result is not zero.		2 or 4	2 or 13...20	2 or 4	2 or 13...28	D		
174	TJZ D,{#}S	Branch S - Test	EEEE 1011100 10I DDDDDDDDD SSSSSSSSS	.	Test D and jump to S** if D is zero.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
175	TJNZ D,{#}S	Branch S - Test	EEEE 1011100 11I DDDDDDDDD SSSSSSSSS	.	Test D and jump to S** if D is not zero.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
176	TJF D,{#}S	Branch S - Test	EEEE 1011101 00I DDDDDDDDD SSSSSSSSS	.	Test D and jump to S** if D is full (D = $FFFF_FFFF).		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
177	TJNF D,{#}S	Branch S - Test	EEEE 1011101 01I DDDDDDDDD SSSSSSSSS	.	Test D and jump to S** if D is not full (D != $FFFF_FFFF).		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
178	TJS D,{#}S	Branch S - Test	EEEE 1011101 10I DDDDDDDDD SSSSSSSSS	.	Test D and jump to S** if D is signed (D[31] = 1).		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
179	TJNS D,{#}S	Branch S - Test	EEEE 1011101 11I DDDDDDDDD SSSSSSSSS	.	Test D and jump to S** if D is not signed (D[31] = 0).		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
180	TJV D,{#}S	Branch S - Test	EEEE 1011110 00I DDDDDDDDD SSSSSSSSS	.	Test D and jump to S** if D overflowed (D[31] != C, C = 'correct sign' from last addition/subtraction).		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
181	JINT {#}S	Events - Branch	EEEE 1011110 01I 000000000 SSSSSSSSS	.	Jump to S** if INT event flag is set.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
182	JCT1 {#}S	Events - Branch	EEEE 1011110 01I 000000001 SSSSSSSSS	.	Jump to S** if CT1 event flag is set.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
183	JCT2 {#}S	Events - Branch	EEEE 1011110 01I 000000010 SSSSSSSSS	.	Jump to S** if CT2 event flag is set.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
184	JCT3 {#}S	Events - Branch	EEEE 1011110 01I 000000011 SSSSSSSSS	.	Jump to S** if CT3 event flag is set.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
185	JSE1 {#}S	Events - Branch	EEEE 1011110 01I 000000100 SSSSSSSSS	.	Jump to S** if SE1 event flag is set.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
186	JSE2 {#}S	Events - Branch	EEEE 1011110 01I 000000101 SSSSSSSSS	.	Jump to S** if SE2 event flag is set.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
187	JSE3 {#}S	Events - Branch	EEEE 1011110 01I 000000110 SSSSSSSSS	.	Jump to S** if SE3 event flag is set.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
188	JSE4 {#}S	Events - Branch	EEEE 1011110 01I 000000111 SSSSSSSSS	.	Jump to S** if SE4 event flag is set.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
189	JPAT {#}S	Events - Branch	EEEE 1011110 01I 000001000 SSSSSSSSS	.	Jump to S** if PAT event flag is set.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
190	JFBW {#}S	Events - Branch	EEEE 1011110 01I 000001001 SSSSSSSSS	.	Jump to S** if FBW event flag is set.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
191	JXMT {#}S	Events - Branch	EEEE 1011110 01I 000001010 SSSSSSSSS	.	Jump to S** if XMT event flag is set.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
192	JXFI {#}S	Events - Branch	EEEE 1011110 01I 000001011 SSSSSSSSS	.	Jump to S** if XFI event flag is set.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
193	JXRO {#}S	Events - Branch	EEEE 1011110 01I 000001100 SSSSSSSSS	.	Jump to S** if XRO event flag is set.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
194	JXRL {#}S	Events - Branch	EEEE 1011110 01I 000001101 SSSSSSSSS	.	Jump to S** if XRL event flag is set.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
195	JATN {#}S	Events - Branch	EEEE 1011110 01I 000001110 SSSSSSSSS	.	Jump to S** if ATN event flag is set.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
196	JQMT {#}S	Events - Branch	EEEE 1011110 01I 000001111 SSSSSSSSS	.	Jump to S** if QMT event flag is set.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
197	JNINT {#}S	Events - Branch	EEEE 1011110 01I 000010000 SSSSSSSSS	.	Jump to S** if INT event flag is clear.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
198	JNCT1 {#}S	Events - Branch	EEEE 1011110 01I 000010001 SSSSSSSSS	.	Jump to S** if CT1 event flag is clear.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
199	JNCT2 {#}S	Events - Branch	EEEE 1011110 01I 000010010 SSSSSSSSS	.	Jump to S** if CT2 event flag is clear.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
200	JNCT3 {#}S	Events - Branch	EEEE 1011110 01I 000010011 SSSSSSSSS	.	Jump to S** if CT3 event flag is clear.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
201	JNSE1 {#}S	Events - Branch	EEEE 1011110 01I 000010100 SSSSSSSSS	.	Jump to S** if SE1 event flag is clear.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
202	JNSE2 {#}S	Events - Branch	EEEE 1011110 01I 000010101 SSSSSSSSS	.	Jump to S** if SE2 event flag is clear.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
203	JNSE3 {#}S	Events - Branch	EEEE 1011110 01I 000010110 SSSSSSSSS	.	Jump to S** if SE3 event flag is clear.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
204	JNSE4 {#}S	Events - Branch	EEEE 1011110 01I 000010111 SSSSSSSSS	.	Jump to S** if SE4 event flag is clear.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
205	JNPAT {#}S	Events - Branch	EEEE 1011110 01I 000011000 SSSSSSSSS	.	Jump to S** if PAT event flag is clear.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
206	JNFBW {#}S	Events - Branch	EEEE 1011110 01I 000011001 SSSSSSSSS	.	Jump to S** if FBW event flag is clear.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
207	JNXMT {#}S	Events - Branch	EEEE 1011110 01I 000011010 SSSSSSSSS	.	Jump to S** if XMT event flag is clear.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
208	JNXFI {#}S	Events - Branch	EEEE 1011110 01I 000011011 SSSSSSSSS	.	Jump to S** if XFI event flag is clear.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
209	JNXRO {#}S	Events - Branch	EEEE 1011110 01I 000011100 SSSSSSSSS	.	Jump to S** if XRO event flag is clear.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
210	JNXRL {#}S	Events - Branch	EEEE 1011110 01I 000011101 SSSSSSSSS	.	Jump to S** if XRL event flag is clear.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
211	JNATN {#}S	Events - Branch	EEEE 1011110 01I 000011110 SSSSSSSSS	.	Jump to S** if ATN event flag is clear.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			
212	JNQMT {#}S	Events - Branch	EEEE 1011110 01I 000011111 SSSSSSSSS	.	Jump to S** if QMT event flag is clear.		2 or 4	2 or 13...20	2 or 4	2 or 13...28			

order	#S = immediate (I=1). S = register. #D = immediate (L=1). D = register. - Assembly Syntax -	- Group -	- Encoding -	- Alias -	* Z = (result == 0). ** If #S and cogex, PC += signed(S). If #S and hubex, PC += signed(S*4). If S, PC = register S. - Description -	Next Inst Shielded from Interrupt	Clock Cycles (8 cogs) * +1 if crosses hub long - Cog Exec Mode - - LUT Exec Mode -	Clock Cycles (8 cogs) * +1 if crosses hub long - Hub Exec Mode -	Clock Cycles (16 cogs) * +1 if crosses hub long - Cog Exec Mode - - LUT Exec Mode -	Clock Cyles (16 cogs) * +1 if crosses hub long - Hub Exec Mode -	* Data not forwarded. - Register Write -	- Hub R/W -	- Stack R/W -
213	<empty> (#)D,{#}S	Miscellaneous	EEEE 1011110 1LI DDDDDDDD SSSSSSSS		<empty>								
214	<empty> (#)D,{#}S	Miscellaneous	EEEE 1011111 0LI DDDDDDDD SSSSSSSS	.	<empty>								
215	SETPAT (#)D,{#}S	Events - Configuration	EEEE 1011111 1LI DDDDDDDD SSSSSSSS	.	Set pin pattern for PAT event. C selects INA/INB, Z selects =/!=, D provides mask value, S provides match value.		2	same	2	same			
216	AKPIN (#)S	Smart Pins	EEEE 1100000 01I 000000001 SSSSSSSS	alias	Acknowledge smart pins S[10:6]+S[5:0]..S[5:0]. Wraps within A/B pins. Prior SETQ D[4:0] overrides S[10:6].		2	same	2	same			
217	WRPIN (#)D,{#}S	Smart Pins	EEEE 1100000 0LI DDDDDDDD SSSSSSSS	.	Set mode of smart pins S[10:6]+S[5:0]..S[5:0] to D, acknowledge pins. Wraps within A/B pins. Prior SETQ D[4:0] overrides S[10:6].		2	same	2	same			
218	WXPIN (#)D,{#}S	Smart Pins	EEEE 1100000 1LI DDDDDDDD SSSSSSSS	.	Set "X" of smart pins S[10:6]+S[5:0]..S[5:0] to D, acknowledge pins. Wraps within A/B pins. Prior SETQ D[4:0] overrides S[10:6].		2	same	2	same			
219	WYPIN (#)D,{#}S	Smart Pins	EEEE 1100001 0LI DDDDDDDD SSSSSSSS	.	Set "Y" of smart pins S[10:6]+S[5:0]..S[5:0] to D, acknowledge pins. Wraps within A/B pins. Prior SETQ D[4:0] overrides S[10:6].		2	same	2	same			
220	WRLUT (#)D,{#}S/P	Lookup Table	EEEE 1100001 1LI DDDDDDDD SSSSSSSS	.	Write D to LUT address {#}S/PTRx.		2	same	2	same			
221	WRBYTE (#)D,{#}S/P	Hub RAM - Write	EEEE 1100010 0LI DDDDDDDD SSSSSSSS	.	Write byte in D[7:0] to hub address {#}S/PTRx.		3...10	3...20	3...18	3...38		Write	
222	WRWORD (#)D,{#}S/P	Hub RAM - Write	EEEE 1100010 1LI DDDDDDDD SSSSSSSS	.	Write word in D[15:0] to hub address {#}S/PTRx.		3...10*	3...20 *	3...18 *	3...38 *		Write	
223	WRLONG (#)D,{#}S/P	Hub RAM - Write	EEEE 1100011 0LI DDDDDDDD SSSSSSSS	.	Write long in D[31:0] to hub address {#}S/PTRx. Prior SETQ/SETQ2 invokes cog/LUT block transfer.		3...10*	3...20 *	3...18 *	3...38 *		Write	
224	PUSHA (#)D	Hub RAM - Write	EEEE 1100011 0LI DDDDDDDD 101100001	alias	Write long in D[31:0] to hub address PTRA++.		3...10*	3...20 *	3...18 *	3...38 *		Write	
225	PUSHB (#)D	Hub RAM - Write	EEEE 1100011 0LI DDDDDDDD 111100001	alias	Write long in D[31:0] to hub address PTRB++.		3...10*	3...20 *	3...18 *	3...38 *		Write	
226	RDFAST {#}D,{#}S	Hub FIFO - New Read	EEEE 1100011 1LI DDDDDDDD SSSSSSSS	.	Begin new fast hub read via FIFO. D[31] = no wait, D[13:0] = block size in 64-byte units (0 = max), S[19:0] = block start address.		2 or WRFAST finish + 10...17	FIFO IN USE	2 or WRFAST finish + 10...25	FIFO IN USE			
227	WRFAST {#}D,{#}S	Hub FIFO - New Write	EEEE 1100100 0LI DDDDDDDD SSSSSSSS	.	Begin new fast hub write via FIFO. D[31] = no wait, D[13:0] = block size in 64-byte units (0 = max), S[19:0] = block start address.		2 or WRFAST finish + 3	FIFO IN USE	2 or WRFAST finish + 3	FIFO IN USE			
228	FBLOCK {#}D,{#}S	Hub FIFO - New Block	EEEE 1100100 1LI DDDDDDDD SSSSSSSS	.	Set next block for when block wraps. D[13:0] = block size in 64-byte units (0 = max), S[19:0] = block start address.		2	FIFO IN USE	2	FIFO IN USE			
229	XINIT {#}D,{#}S	Streamer	EEEE 1100101 0LI DDDDDDDD SSSSSSSS	.	Issue streamer command immediately, zeroing phase. Prior SETQ sets NCO frequency.		2	same	2	same			
230	XSTOP	Streamer	EEEE 1100101 011 000000000 000000000	alias	Stop streamer immediately. Prior SETQ sets NCO frequency.		2	same	2	same			
231	XZERO {#}D,{#}S	Streamer	EEEE 1100101 1LI DDDDDDDD SSSSSSSS	.	Buffer new streamer command to be issued on final NCO rollover of current command, zeroing phase. Prior SETQ sets NCO frequency.		2+	same	2+	same			
232	XCONT {#}D,{#}S	Streamer	EEEE 1100110 0LI DDDDDDDD SSSSSSSS	.	Buffer new streamer command to be issued on final NCO rollover of current command, continuing phase. Prior SETQ sets NCO frequency.		2+	same	2+	same			
233	REP {#}D,{#}S	Branch Repeat	EEEE 1100110 1LI DDDDDDDD SSSSSSSS	.	Execute next D[8:0] instructions S times. If S = 0, repeat instructions infinitely. If D[8:0] = 0, nothing repeats.	✔*code	2	same	2	same			
234	COGINIT {#}D,{#}S {WC}	Hub Control - Cogs	EEEE 1100111 CLI DDDDDDDD SSSSSSSS	.	Start cog selected by D. S[19:0] sets hub startup address and PTRB of cog. Prior SETQ sets PTRA of cog. C = 1 if no cog available.		2...9, +2 if result	same	2...17, +2 if result	same	D if reg and WC		
235	QMUL {#}D,{#}S	CORDIC Solver	EEEE 1101000 0LI DDDDDDDD SSSSSSSS	.	Begin CORDIC unsigned multiplication of D * S. GETQX/GETQY retrieves lower/upper product.		2...9	same	2...17	same			
236	QDIV {#}D,{#}S	CORDIC Solver	EEEE 1101000 1LI DDDDDDDD SSSSSSSS	.	Begin CORDIC unsigned division of {SETQ value or 32'b0, D} / S. GETQX/GETQY retrieves quotient/remainder.		2...9	same	2...17	same			
237	QFRAC {#}D,{#}S	CORDIC Solver	EEEE 1101001 0LI DDDDDDDD SSSSSSSS	.	Begin CORDIC unsigned division of {D, SETQ value or 32'b0} / S. GETQX/GETQY retrieves quotient/remainder.		2...9	same	2...17	same			
238	QSQRT {#}D,{#}S	CORDIC Solver	EEEE 1101001 1LI DDDDDDDD SSSSSSSS	.	Begin CORDIC square root of {S, D}. GETQX retrieves root.		2...9	same	2...17	same			
239	QROTATE {#}D,{#}S	CORDIC Solver	EEEE 1101010 0LI DDDDDDDD SSSSSSSS	.	Begin CORDIC rotation of point {D, SETQ value or 32'b0} by angle S. GETQX/GETQY retrieves X/Y.		2...9	same	2...17	same			
240	QVECTOR {#}D,{#}S	CORDIC Solver	EEEE 1101010 1LI DDDDDDDD SSSSSSSS	.	Begin CORDIC vectoring of point {D, S}. GETQX/GETQY retrieves length/angle.		2...9	same	2...17	same			
241	HUBSET {#}D	Hub Control - Multi	EEEE 1101011 00L DDDDDDDD 000000000	.	Set hub configuration to D.		2...9	same	2...17	same			
242	COGID {#}D {WC}	Hub Control - Cogs	EEEE 1101011 00L DDDDDDDD 000000001	.	If D is register and no WC, get cog ID (0 to 15) into D. If WC, check status of cog D[3:0], C = 1 if on.		2...9, +2 if result	same	2...17, +2 if result	same	D if reg and !WC		
243	COGSTOP {#}D	Hub Control - Cogs	EEEE 1101011 00L DDDDDDDD 000000011	.	Stop cog D[3:0].		2...9	same	2...17	same			
244	LOCKNEW D {WC}	Hub Control - Locks	EEEE 1101011 00L DDDDDDDD 000000100	.	Request a LOCK. D will be written with the LOCK number (0 to 15). C = 1 if no LOCK available.		4...11	same	4...19	same	D		
245	LOCKRET {#}D	Hub Control - Locks	EEEE 1101011 00L DDDDDDDD 000000101	.	Return LOCK D[3:0] for reallocation.		2...9	same	2...17	same			
246	LOCKTRY {#}D {WC}	Hub Control - Locks	EEEE 1101011 00L DDDDDDDD 000000110	.	Try to get LOCK D[3:0]. C = 1 if got LOCK. LOCKREL releases LOCK. LOCK is also released if owner cog stops or restarts.		2...9, +2 if result	same	2...17, +2 if result	same			
247	LOCKREL {#}D {WC}	Hub Control - Locks	EEEE 1101011 00L DDDDDDDD 000000111	.	Release LOCK D[3:0]. If D is a register and WC, get current/last cog ID of LOCK owner into D and LOCK status into C.		2...9, +2 if result	same	2...17, +2 if result	same			
248	QLOG {#}D	CORDIC Solver	EEEE 1101011 00L DDDDDDDD 000001110	.	Begin CORDIC number-to-logarithm conversion of D. GETQX retrieves log (5'whole_exponent, 27'fractional_exponent).		2...9	same	2...17	same			
249	QEXP {#}D	CORDIC Solver	EEEE 1101011 00L DDDDDDDD 000001111	.	Begin CORDIC logarithm-to-number conversion of D. GETQX retrieves number.		2...9	same	2...17	same			
250	RFBYTE D {WC/WZ/WCZ}	Hub FIFO - Read	EEEE 1101011 C00 DDDDDDDD 000010000	.	Used after RDFAST. Read zero-extended byte from FIFO into D. C = MSB of byte. *		2	FIFO IN USE	2	FIFO IN USE	D	Read	
251	RFWORD D {WC/WZ/WCZ}	Hub FIFO - Read	EEEE 1101011 C00 DDDDDDDD 000010001	.	Used after RDFAST. Read zero-extended word from FIFO into D. C = MSB of word. *		2	FIFO IN USE	2	FIFO IN USE	D	Read	
252	RFLONG D {WC/WZ/WCZ}	Hub FIFO - Read	EEEE 1101011 C00 DDDDDDDD 000010010	.	Used after RDFAST. Read long from FIFO into D. C = MSB of long. *		2	FIFO IN USE	2	FIFO IN USE	D	Read	
253	RFVAR D {WC/WZ/WCZ}	Hub FIFO - Read	EEEE 1101011 C00 DDDDDDDD 000010011	.	Used after RDFAST. Read zero-extended 1..4-byte value from FIFO into D. C = 0. *		2	FIFO IN USE	2	FIFO IN USE	D	Read	
254	RFVARS D {WC/WZ/WCZ}	Hub FIFO - Read	EEEE 1101011 C00 DDDDDDDD 000010100	.	Used after RDFAST. Read sign-extended 1..4-byte value from FIFO into D. C = MSB of value. *		2	FIFO IN USE	2	FIFO IN USE	D	Read	
255	WFBYTE {#}D	Hub FIFO - Write	EEEE 1101011 00L DDDDDDDD 000010101	.	Used after WRFAST. Write byte in D[7:0] into FIFO.		2	FIFO IN USE	2	FIFO IN USE		Write	
256	WFWORD {#}D	Hub FIFO - Write	EEEE 1101011 00L DDDDDDDD 000010110	.	Used after WRFAST. Write word in D[15:0] into FIFO.		2	FIFO IN USE	2	FIFO IN USE		Write	
257	WFLONG {#}D	Hub FIFO - Write	EEEE 1101011 00L DDDDDDDD 000010111	.	Used after WRFAST. Write long in D[31:0] into FIFO.		2	FIFO IN USE	2	FIFO IN USE		Write	
258	GETQX D {WC/WZ/WCZ}	CORDIC Solver	EEEE 1101011 C00 DDDDDDDD 000011000	.	Retrieve CORDIC result X into D. Waits, in case result not ready. C = X[31]. *		2...58	same	2...58	same	D		
259	GETQY D {WC/WZ/WCZ}	CORDIC Solver	EEEE 1101011 C00 DDDDDDDD 000011001	.	Retrieve CORDIC result Y into D. Waits, in case result not ready. C = Y[31]. *		2...58	same	2...58	same	D		
260	GETCT D {WC}	Miscellaneous	EEEE 1101011 C00 DDDDDDDD 000011010	.	Get CT[31:0] or CT[63:32] if WC into D. GETCT WC + GETCT captures entire CT. CT=0 on reset, CT++ on every clock. C = same.	✔ if WC	2	same	2	same	D		
261	GETRND D {WC/WZ/WCZ}	Miscellaneous	EEEE 1101011 C00 DDDDDDDD 000011011	.	Get RND into D/C/Z. RND is the PRNG that updates on every clock. D = RND[31:0], C = RND[31], Z = RND[30], unique per cog.		2	same	2	same	D		
262	GETRND WC/WZ/WCZ	Miscellaneous	EEEE 1101011 C10 000000000 000011011	alias	Get RND into C/Z. C = RND[31], Z = RND[30], unique per cog.		2	same	2	same			
263	SETDAC5 {#}D	Smart Pins	EEEE 1101011 00L DDDDDDDD 000011100	.	DAC3 = D[31:24], DAC2 = D[23:16], DAC1 = D[15:8], DAC0 = D[7:0].		2	same	2	same			
264	SETXFRQ {#}D	Streamer	EEEE 1101011 00L DDDDDDDD 000011101	.	Set streamer NCO frequency to D.		2	same	2	same			
265	GETXACC D	Streamer	EEEE 1101011 000 DDDDDDDD 000011110	.	Get the streamer's Goertzel X accumulator into D and the Y accumulator into the next instruction's S, clear accumulators.	✔	2	same	2	same	D		
266	WAITX {#}D {WC/WZ/WCZ}	Miscellaneous	EEEE 1101011 C0L DDDDDDDD 000011111	.	Wait 2 + D clocks if no WC/WZ/WCZ. If WC/WZ/WCZ, wait 2 + (D & RND) clocks. C/Z = 0.		2 + D	same	2 + D	same			
267	SETSE1 {#}D	Events - Configuration	EEEE 1101011 00L DDDDDDDD 000100000	.	Set SE1 event configuration to D[8:0].		2	same	2	same			
268	SETSE2 {#}D	Events - Configuration	EEEE 1101011 00L DDDDDDDD 000100001	.	Set SE2 event configuration to D[8:0].		2	same	2	same			
269	SETSE3 {#}D	Events - Configuration	EEEE 1101011 00L DDDDDDDD 000100010	.	Set SE3 event configuration to D[8:0].		2	same	2	same			
270	SETSE4 {#}D	Events - Configuration	EEEE 1101011 00L DDDDDDDD 000100011	.	Set SE4 event configuration to D[8:0].		2	same	2	same			
271	POLLINT {WC/WZ/WCZ}	Events - Poll	EEEE 1101011 C20 000000000 000100100	.	Get INT event flag into C/Z, then clear it.		2	same	2	same			
272	POLLCT1 {WC/WZ/WCZ}	Events - Poll	EEEE 1101011 C20 000000001 000100100	.	Get CT1 event flag into C/Z, then clear it.		2	same	2	same			
273	POLLCT2 {WC/WZ/WCZ}	Events - Poll	EEEE 1101011 C20 000000010 000100100	.	Get CT2 event flag into C/Z, then clear it.		2	same	2	same			
274	POLLCT3 {WC/WZ/WCZ}	Events - Poll	EEEE 1101011 C20 000000011 000100100	.	Get CT3 event flag into C/Z, then clear it.		2	same	2	same			
275	POLLSE1 {WC/WZ/WCZ}	Events - Poll	EEEE 1101011 C20 000000100 000100100	.	Get SE1 event flag into C/Z, then clear it.		2	same	2	same			
276	POLLSE2 {WC/WZ/WCZ}	Events - Poll	EEEE 1101011 C20 000000101 000100100	.	Get SE2 event flag into C/Z, then clear it.		2	same	2	same			
277	POLLSE3 {WC/WZ/WCZ}	Events - Poll	EEEE 1101011 C20 000000110 000100100	.	Get SE3 event flag into C/Z, then clear it.		2	same	2	same			
278	POLLSE4 {WC/WZ/WCZ}	Events - Poll	EEEE 1101011 C20 000000111 000100100	.	Get SE4 event flag into C/Z, then clear it.		2	same	2	same			
279	POLLPAT {WC/WZ/WCZ}	Events - Poll	EEEE 1101011 C20 000001000 000100100	.	Get PAT event flag into C/Z, then clear it.		2	same	2	same			
280	POLLFBW {WC/WZ/WCZ}	Events - Poll	EEEE 1101011 C20 000001001 000100100	.	Get FBW event flag into C/Z, then clear it.		2	same	2	same			
281	POLLXMT {WC/WZ/WCZ}	Events - Poll	EEEE 1101011 C20 000001010 000100100	.	Get XMT event flag into C/Z, then clear it.		2	same	2	same			
282	POLLXFI {WC/WZ/WCZ}	Events - Poll	EEEE 1101011 C20 000001011 000100100	.	Get XFI event flag into C/Z, then clear it.		2	same	2	same			
283	POLLXRO {WC/WZ/WCZ}	Events - Poll	EEEE 1101011 C20 000001100 000100100	.	Get XRO event flag into C/Z, then clear it.		2	same	2	same			
284	POLLXRL {WC/WZ/WCZ}	Events - Poll	EEEE 1101011 C20 000001101 000100100	.	Get XRL event flag into C/Z, then clear it.		2	same	2	same			
285	POLLATN {WC/WZ/WCZ}	Events - Poll	EEEE 1101011 C20 000001110 000100100	.	Get ATN event flag into C/Z, then clear it.		2	same	2	same			
286	POLLQMT {WC/WZ/WCZ}	Events - Poll	EEEE 1101011 C20 000001111 000100100	.	Get QMT event flag into C/Z, then clear it.		2	same	2	same			
287	WAITINT {WC/WZ/WCZ}	Events - Wait	EEEE 1101011 C20 000010000 000100100	.	Wait for INT event flag, then clear it. Prior SETQ sets optional CT timeout value. C/Z = timeout.		2+	same	2+	same			
288	WAITCT1 {WC/WZ/WCZ}	Events - Wait	EEEE 1101011 C20 000010001 000100100	.	Wait for CT1 event flag, then clear it. Prior SETQ sets optional CT timeout value. C/Z = timeout.		2+	same	2+	same			
289	WAITCT2 {WC/WZ/WCZ}	Events - Wait	EEEE 1101011 C20 000010010 000100100	.	Wait for CT2 event flag, then clear it. Prior SETQ sets optional CT timeout value. C/Z = timeout.		2+	same	2+	same			
290	WAITCT3 {WC/WZ/WCZ}	Events - Wait	EEEE 1101011 C20 000010011 000100100	.	Wait for CT3 event flag, then clear it. Prior SETQ sets optional CT timeout value. C/Z = timeout.		2+	same	2+	same			
291	WAITSE1 {WC/WZ/WCZ}	Events - Wait	EEEE 1101011 C20 000010100 000100100	.	Wait for SE1 event flag, then clear it. Prior SETQ sets optional CT timeout value. C/Z = timeout.		2+	same	2+	same			
292	WAITSE2 {WC/WZ/WCZ}	Events - Wait	EEEE 1101011 C20 000010101 000100100	.	Wait for SE2 event flag, then clear it. Prior SETQ sets optional CT timeout value. C/Z = timeout.		2+	same	2+	same			
293	WAITSE3 {WC/WZ/WCZ}	Events - Wait	EEEE 1101011 C20 000010110 000100100	.	Wait for SE3 event flag, then clear it. Prior SETQ sets optional CT timeout value. C/Z = timeout.		2+	same	2+	same			
294	WAITSE4 {WC/WZ/WCZ}	Events - Wait	EEEE 1101011 C20 000010111 000100100	.	Wait for SE4 event flag, then clear it. Prior SETQ sets optional CT timeout value. C/Z = timeout.		2+	same	2+	same			
295	WAITPAT {WC/WZ/WCZ}	Events - Wait	EEEE 1101011 C20 000011000 000100100	.	Wait for PAT event flag, then clear it. Prior SETQ sets optional CT timeout value. C/Z = timeout.		2+	same	2+	same			
296	WAITFBW {WC/WZ/WCZ}	Events - Wait	EEEE 1101011 C20 000011001 000100100	.	Wait for FBW event flag, then clear it. Prior SETQ sets optional CT timeout value. C/Z = timeout.		2+	same	2+	same			
297	WAITXMT {WC/WZ/WCZ}	Events - Wait	EEEE 1101011 C20 000011010 000100100	.	Wait for XMT event flag, then clear it. Prior SETQ sets optional CT timeout value. C/Z = timeout.		2+	same	2+	same			
298	WAITXFI {WC/WZ/WCZ}	Events - Wait	EEEE 1101011 C20 000011011 000100100	.	Wait for XFI event flag, then clear it. Prior SETQ sets optional CT timeout value. C/Z = timeout.		2+	same	2+	same			
299	WAITXRO {WC/WZ/WCZ}	Events - Wait	EEEE 1101011 C20 000011100 000100100	.	Wait for XRO event flag, then clear it. Prior SETQ sets optional CT timeout value. C/Z = timeout.		2+	same	2+	same			
300	WAITXRL {WC/WZ/WCZ}	Events - Wait	EEEE 1101011 C20 000011101 000100100	.	Wait for XRL event flag, then clear it. Prior SETQ sets optional CT timeout value. C/Z = timeout.		2+	same	2+	same			
301	WAITATN {WC/WZ/WCZ}	Events - Wait	EEEE 1101011 C20 000011110 000100100	.	Wait for ATN event flag, then clear it. Prior SETQ sets optional CT timeout value. C/Z = timeout.		2+	same	2+	same			
302	ALLOWI	Interrupts	EEEE 1101011 000 000100000 000100100	.	Allow interrupts (default).		2	same	2	same			
303	STALLI	Interrupts	EEEE 1101011 000 000100001 000100100	.	Stall Interrupts.		2	same	2	same			
304	TRGINT1	Interrupts	EEEE 1101011 000 000100010 000100100	.	Trigger INT1, regardless of STALLI mode.		2	same	2	same			
305	TRGINT2	Interrupts	EEEE 1101011 000 000100011 000100100	.	Trigger INT2, regardless of STALLI mode.		2	same	2	same			
306	TRGINT3	Interrupts	EEEE 1101011 000 000100100 000100100	.	Trigger INT3, regardless of STALLI mode.		2	same	2	same			
307	NIXINT1	Interrupts	EEEE 1101011 000 000100101 000100100	.	Cancel INT1.		2	same	2	same			
308	NIXINT2	Interrupts	EEEE 1101011 000 000100110 000100100	.	Cancel INT2.		2	same	2	same			
309	NIXINT3	Interrupts	EEEE 1101011 000 000100111 000100100	.	Cancel INT3.		2	same	2	same			
310	SETINT1 {#}D	Interrupts	EEEE 1101011 00L DDDDDDDD 000100101	.	Set INT1 source to D[3:0].		2	same	2	same			
311	SETINT2 {#}D	Interrupts	EEEE 1101011 00L DDDDDDDD 000100110	.	Set INT2 source to D[3:0].		2	same	2	same			
312	SETINT3 {#}D	Interrupts	EEEE 1101011 00L DDDDDDDD 000100111	.	Set INT3 source to D[3:0].		2	same	2	same			
313	SETQ {#}D	Miscellaneous	EEEE 1101011 00L DDDDDDDD 000101000	.	Set Q to D. Use before RDLONG/WRLONG/WMLONG to set block transfer. Also used before MUXQ/COGINIT/QDIV/QFRAC/QROTATE/WAITxxx.	✔	2	same	2	same			
314	SETQ2 {#}D	Miscellaneous	EEEE 1101011 00L DDDDDDDD 000101001	.	Set Q to D. Use before RDLONG/WRLONG/WMLONG to set LUT block transfer.	✔	2	same	2	same			
315	PUSH {#}D	Miscellaneous	EEEE 1101011 00L DDDDDDDD 000101010	.	Push D onto stack.		2	same	2	same			Push
316	POP D {WC/WZ/WCZ}	Miscellaneous	EEEE 1101011 C20 DDDDDDDD 000101011	.	Pop stack {K}. D = K. C = K[31]. *		2	same	2	same	D		Pop
317	JMP D {WC/WZ/WCZ}	Branch D - Jump	EEEE 1101011 C20 DDDDDDDD 000101100	.	Jump to D. C = D[31], Z = D[30], PC = D[19:0].		4	13...20	4	13...28			
318	CALL D {WC/WZ/WCZ}	Branch D - Call	EEEE 1101011 C20 DDDDDDDD 000101101	.	Call to D by pushing {C, Z, 10'b0, PC[19:0]} onto stack. C = D[31], Z = D[30], PC = D[19:0].		4	13...20	4	13...28			Push

#S = immediate (I=1). S = register.
#D = immediate (L=1). D = register.

* Z = (result == 0).
** If #S and cogex, PC += signed(S). If #S and hubex, PC += signed(S*4). If S, PC = register S.

order	- Assembly Syntax -			- Group -	- Encoding -	- Alias -	- Description -	Next Inst Shielded from Interrupt	Clock Cycles (8 cogs) * +1 if crosses hub long - Cog Exec Mode - - LUT Exec Mode -	Clock Cycles (8 cogs) * +1 if crosses hub long - Hub Exec Mode -	Clock Cycles (16 cogs) * +1 if crosses hub long - Cog Exec Mode - - LUT Exec Mode -	Clock Cycles (16 cogs) * +1 if crosses hub long - Hub Exec Mode -	* Data not forwarded. - Register Write -	- Hub R/W -	- Stack R/W -
319	RET		(WC/WZ/WCZ)	Branch Return	EEEE 1101011 CZ1 000000000 000101101	.	Return by popping stack (K). C = K[31], Z = K[30], PC = K[19:0].		4	13...20	4	13...28			Pop
320	CALLA	D	(WC/WZ/WCZ)	Branch D - Call	EEEE 1101011 CZ0 DDDDDDDDD 000101110	.	Call to D by writing {C, Z, 10'b0, PC[19:0]} to hub long at PTRA++. C = D[31], Z = D[30], PC = D[19:0].		5...12 *	14...32 *	5...20 *	14...64 *		Write	
321	RETA		(WC/WZ/WCZ)	Branch Return	EEEE 1101011 CZ1 000000000 000101110	.	Return by reading hub long (L) at --PTRA. C = L[31], Z = L[30], PC = L[19:0].		11...18 *	20...40 *	11...26 *	20...70 *		Read	
322	CALLB	D	(WC/WZ/WCZ)	Branch D - Call	EEEE 1101011 CZ0 DDDDDDDDD 000101111	.	Call to D by writing {C, Z, 10'b0, PC[19:0]} to hub long at PTRB++. C = D[31], Z = D[30], PC = D[19:0].		5...12 *	14...32 *	5...20 *	14...64 *		Write	
323	RETB		(WC/WZ/WCZ)	Branch Return	EEEE 1101011 CZ1 000000000 000101111	.	Return by reading hub long (L) at --PTRB. C = L[31], Z = L[30], PC = L[19:0].		11...18 *	20...40 *	11...26 *	20...70 *		Read	
324	JMPREL	{#}D		Branch D - Jump	EEEE 1101011 00L DDDDDDDDD 000110000	.	Jump ahead/back by D instructions. For cogex, PC += D[19:0]. For hubex, PC += D[17:0] << 2.		4	13...20	4	13...28			
325	SKIP	{#}D		Branch D - Skip	EEEE 1101011 00L DDDDDDDDD 000110001	.	Skip instructions per D. Subsequent instructions 0..31 get cancelled for each '1' bit in D[0]..D[31].		2	same	2	same			
326	SKIPF	{#}D		Branch D - Jump+Skip	EEEE 1101011 00L DDDDDDDDD 000110010	.	Skip cog/LUT instructions fast per D. Like SKIP, but instead of cancelling instructions, the PC leaps over them.		2	ILLEGAL	2	ILLEGAL			
327	EXECF	{#}D		Branch D - Call+Skip	EEEE 1101011 00L DDDDDDDDD 000110011	.	Jump to D[9:0] in cog/LUT and set SKIPF pattern to D[31:i0]. PC = {10'b0, D[9:0]}.		4	same	4	same			
328	GETPTR	D		Hub FIFO	EEEE 1101011 000 DDDDDDDDD 000110100	.	Get current FIFO hub pointer into D.		2	FIFO IN USE	2	FIFO IN USE	D		
329	GETBRK	D	WC/WZ/WCZ	Interrupts	EEEE 1101011 CZ0 DDDDDDDDD 000110101	.	Get breakpoint/cog status into D according to WC/WZ/WCZ. See documentation for details.		2	same	2	same	D		
330	COGBRK	{#}D		Interrupts	EEEE 1101011 00L DDDDDDDDD 000110101	.	If in debug ISR, trigger asynchronous breakpoint in cog D[3:0]. Cog D[3:0] must have asynchronous breakpoint enabled.		2	same	2	same			
331	BRK	{#}D		Interrupts	EEEE 1101011 00L DDDDDDDDD 000110110	.	If in debug ISR, set next break condition to D. Else, set BRK code to D[7:0] and unconditionally trigger BRK interrupt, if enabled.		2	same	2	same			
332	SETLUTS	{#}D		Lookup Table	EEEE 1101011 00L DDDDDDDDD 000110111	.	If D[0] = 1 then enable LUT sharing, where LUT writes within the adjacent odd/even companion cog are copied to this cog's LUT.		2	same	2	same			
333	SETCY	{#}D		Color Space Converter	EEEE 1101011 00L DDDDDDDDD 000111000	.	Set the colorspace converter "CY" parameter to D[31:0].		2	same	2	same			
334	SETCI	{#}D		Color Space Converter	EEEE 1101011 00L DDDDDDDDD 000111001	.	Set the colorspace converter "CI" parameter to D[31:0].		2	same	2	same			
335	SETCQ	{#}D		Color Space Converter	EEEE 1101011 00L DDDDDDDDD 000111010	.	Set the colorspace converter "CQ" parameter to D[31:0].		2	same	2	same			
336	SETCFRQ	{#}D		Color Space Converter	EEEE 1101011 00L DDDDDDDDD 000111011	.	Set the colorspace converter "CFRQ" parameter to D[31:0].		2	same	2	same			
337	SETCMOD	{#}D		Color Space Converter	EEEE 1101011 00L DDDDDDDDD 000111100	.	Set the colorspace converter "CMOD" parameter to D[8:0].		2	same	2	same			
338	SETPIV	{#}D		Pixel Mixer	EEEE 1101011 00L DDDDDDDDD 000111101	.	Set BLNPIX/MIXPIX blend factor to D[7:0].		2	same	2	same			
339	SETPIX	{#}D		Pixel Mixer	EEEE 1101011 00L DDDDDDDDD 000111110	.	Set MIXPIX mode to D[5:0].		2	same	2	same			
340	COGATN	{#}D		Events - Attention	EEEE 1101011 00L DDDDDDDDD 000111111	.	Strobe "attention" of all cogs whose corresponding bits are high in D[15:0].		2	same	2	same			
341	TESTP	{#}D	WC/WZ	Pins	EEEE 1101011 CZL DDDDDDDDD 001000000	.	Test IN bit of pin D[5:0], write to C/Z. C/Z = IN[D[5:0]].		2	same	2	same			
342	TESTPN	{#}D	WC/WZ	Pins	EEEE 1101011 CZL DDDDDDDDD 001000001	.	Test !IN bit of pin D[5:0], write to C/Z. C/Z = !IN[D[5:0]].		2	same	2	same			
343	TESTP	{#}D	ANDC/ANDZ	Pins	EEEE 1101011 CZL DDDDDDDDD 001000010	.	Test IN bit of pin D[5:0], AND into C/Z. C/Z = C/Z AND IN[D[5:0]].		2	same	2	same			
344	TESTPN	{#}D	ANDC/ANDZ	Pins	EEEE 1101011 CZL DDDDDDDDD 001000011	.	Test !IN bit of pin D[5:0], AND into C/Z. C/Z = C/Z AND !IN[D[5:0]].		2	same	2	same			
345	TESTP	{#}D	ORC/ORZ	Pins	EEEE 1101011 CZL DDDDDDDDD 001000100	.	Test IN bit of pin D[5:0], OR into C/Z. C/Z = C/Z OR IN[D[5:0]].		2	same	2	same			
346	TESTPN	{#}D	ORC/ORZ	Pins	EEEE 1101011 CZL DDDDDDDDD 001000101	.	Test !IN bit of pin D[5:0], OR into C/Z. C/Z = C/Z OR !IN[D[5:0]].		2	same	2	same			
347	TESTP	{#}D	XORC/XORZ	Pins	EEEE 1101011 CZL DDDDDDDDD 001000110	.	Test IN bit of pin D[5:0], XOR into C/Z. C/Z = C/Z XOR IN[D[5:0]].		2	same	2	same			
348	TESTPN	{#}D	XORC/XORZ	Pins	EEEE 1101011 CZL DDDDDDDDD 001000111	.	Test !IN bit of pin D[5:0], XOR into C/Z. C/Z = C/Z XOR !IN[D[5:0]].		2	same	2	same			
349	DIRL	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001000000	.	DIR bits of pins D[10:6]+D[5:0]..D[5:0] = 0. Wraps within DIRA/DIRB. Prior SETQ overrides D[10:6]. C,Z = DIR[D[5:0]].		2	same	2	same	DIRx		
350	DIRH	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001000001	.	DIR bits of pins D[10:6]+D[5:0]..D[5:0] = 1. Wraps within DIRA/DIRB. Prior SETQ overrides D[10:6]. C,Z = DIR[D[5:0]].		2	same	2	same	DIRx		
351	DIRC	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001000010	.	DIR bits of pins D[10:6]+D[5:0]..D[5:0] = C. Wraps within DIRA/DIRB. Prior SETQ overrides D[10:6]. C,Z = DIR[D[5:0]].		2	same	2	same	DIRx		
352	DIRNC	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001000011	.	DIR bits of pins D[10:6]+D[5:0]..D[5:0] = !C. Wraps within DIRA/DIRB. Prior SETQ overrides D[10:6]. C,Z = DIR[D[5:0]].		2	same	2	same	DIRx		
353	DIRZ	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001000100	.	DIR bits of pins D[10:6]+D[5:0]..D[5:0] = Z. Wraps within DIRA/DIRB. Prior SETQ overrides D[10:6]. C,Z = DIR[D[5:0]].		2	same	2	same	DIRx		
354	DIRNZ	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001000101	.	DIR bits of pins D[10:6]+D[5:0]..D[5:0] = !Z. Wraps within DIRA/DIRB. Prior SETQ overrides D[10:6]. C,Z = DIR[D[5:0]].		2	same	2	same	DIRx		
355	DIRRND	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001000110	.	DIR bits of pins D[10:6]+D[5:0]..D[5:0] = RNDa. Wraps within DIRA/DIRB. Prior SETQ overrides D[10:6]. C,Z = DIR[D[5:0]].		2	same	2	same	DIRx		
356	DIRNOT	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001000111	.	Toggle DIR bits of pins D[10:6]+D[5:0]..D[5:0]. Wraps within DIRA/DIRB. Prior SETQ overrides D[10:6]. C,Z = DIR[D[5:0]].		2	same	2	same	DIRx		
357	OUTL	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001001000	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = 0. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	OUTx		
358	OUTH	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001001001	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = 1. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	OUTx		
359	OUTC	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001001010	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = C. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	OUTx		
360	OUTNC	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001001011	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = !C. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	OUTx		
361	OUTZ	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001001100	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = Z. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	OUTx		
362	OUTNZ	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001001101	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = !Z. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	OUTx		
363	OUTRND	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001001110	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = RNDa. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	OUTx		
364	OUTNOT	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001001111	.	Toggle OUT bits of pins D[10:6]+D[5:0]..D[5:0]. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	OUTx		
365	FLTL	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001010000	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = 0. DIR bits = 0. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	DIRx* + OUTx		
366	FLTH	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001010001	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = 1. DIR bits = 0. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	DIRx* + OUTx		
367	FLTC	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001010010	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = C. DIR bits = 0. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	DIRx* + OUTx		
368	FLTNC	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001010011	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = !C. DIR bits = 0. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	DIRx* + OUTx		
369	FLTZ	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001010100	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = Z. DIR bits = 0. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	DIRx* + OUTx		
370	FLTNZ	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001010101	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = !Z. DIR bits = 0. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	DIRx* + OUTx		
371	FLTRND	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001010110	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = RNDa. DIR bits = 0. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	DIRx* + OUTx		
372	FLTNOT	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001010111	.	Toggle OUT bits of pins D[10:6]+D[5:0]..D[5:0]. DIR bits = 0. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	DIRx* + OUTx		
373	DRVL	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001011000	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = 0. DIR bits = 1. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	DIRx* + OUTx		
374	DRVH	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001011001	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = 1. DIR bits = 1. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	DIRx* + OUTx		
375	DRVC	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001011010	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = C. DIR bits = 1. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	DIRx* + OUTx		
376	DRVNC	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001011011	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = !C. DIR bits = 1. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	DIRx* + OUTx		
377	DRVZ	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001011100	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = Z. DIR bits = 1. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	DIRx* + OUTx		
378	DRVNZ	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001011101	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = !Z. DIR bits = 1. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	DIRx* + OUTx		
379	DRVRND	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001011110	.	OUT bits of pins D[10:6]+D[5:0]..D[5:0] = RNDa. DIR bits = 1. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	DIRx* + OUTx		
380	DRVNOT	{#}D	(WCZ)	Pins	EEEE 1101011 CZL DDDDDDDDD 001011111	.	Toggle OUT bits of pins D[10:6]+D[5:0]..D[5:0]. DIR bits = 1. Wraps within OUTA/OUTB. Prior SETQ overrides D[10:6]. C,Z = OUT[D[5:0]].		2	same	2	same	DIRx* + OUTx		
381	SPLITB	D		Math and Logic	EEEE 1101011 000 DDDDDDDDD 001100000	.	Split every 4th bit of D into bytes. D = {D[31], D[27], D[23], D[19], ...D[12], D[8], D[4], D[0]}.		2	same	2	same	D		
382	MERGEB	D		Math and Logic	EEEE 1101011 000 DDDDDDDDD 001100001	.	Merge bits of bytes in D. D = {D[31], D[23], D[15], D[7], ...D[24], D[16], D[8], D[0]}.		2	same	2	same	D		
383	SPLITW	D		Math and Logic	EEEE 1101011 000 DDDDDDDDD 001100010	.	Split odd/even bits of D into words. D = {D[31], D[29], D[27], D[25], ...D[6], D[4], D[2], D[0]}.		2	same	2	same	D		
384	MERGEW	D		Math and Logic	EEEE 1101011 000 DDDDDDDDD 001100011	.	Merge bits of words in D. D = {D[31], D[15], D[30], D[14], ...D[17], D[1], D[16], D[0]}.		2	same	2	same	D		
385	SEUSSF	D		Math and Logic	EEEE 1101011 000 DDDDDDDDD 001100100	.	Relocate and periodically invert bits within D. Returns to original value on 32nd iteration. Forward pattern.		2	same	2	same	D		
386	SEUSSR	D		Math and Logic	EEEE 1101011 000 DDDDDDDDD 001100101	.	Relocate and periodically invert bits within D. Returns to original value on 32nd iteration. Reverse pattern.		2	same	2	same	D		
387	RGBSQZ	D		Math and Logic	EEEE 1101011 000 DDDDDDDDD 001100110	.	Squeeze 8:8:8 RGB value in D[31:8] into 5:6:5 value in D[15:0]. D = {15'b0, D[31:27], D[23:18], D[15:11]}.		2	same	2	same	D		
388	RGBEXP	D		Math and Logic	EEEE 1101011 000 DDDDDDDDD 001100111	.	Expand 5:6:5 RGB value in D[15:0] into 8:8:8 value in D[31:8]. D = {D[15:11,15:13], D[10:5,10:9], D[4:0,4:2], 8'b0}.		2	same	2	same	D		
389	XORO32	D		Math and Logic	EEEE 1101011 000 DDDDDDDDD 001101000	.	Iterate D with xoroshiro32+ PRNG algorithm and put PRNG result into next instruction's S. D must be non-zero to iterate.	✔	2	same	2	same	D		
390	REV	D		Math and Logic	EEEE 1101011 000 DDDDDDDDD 001101001	.	Reverse D bits. D = D[0:31].		2	same	2	same	D		
391	RCZR	D	(WC/WZ/WCZ)	Math and Logic	EEEE 1101011 CZ0 DDDDDDDDD 001101010	.	Rotate C,Z right through D. D = {C, Z, D[31:2]}. C = D[1], Z = D[0].		2	same	2	same	D		
392	RCZL	D	(WC/WZ/WCZ)	Math and Logic	EEEE 1101011 CZ0 DDDDDDDDD 001101011	.	Rotate C,Z left through D. D = {D[29:0], C, Z}. C = D[31], Z = D[30].		2	same	2	same	D		
393	WRC	D		Math and Logic	EEEE 1101011 000 DDDDDDDDD 001101100	.	Write 0 or 1 to D, according to C. D = {31'b0, C}.		2	same	2	same	D		
394	WRNC	D		Math and Logic	EEEE 1101011 000 DDDDDDDDD 001101101	.	Write 0 or 1 to D, according to !C. D = {31'b0, !C}.		2	same	2	same	D		
395	WRZ	D		Math and Logic	EEEE 1101011 000 DDDDDDDDD 001101110	.	Write 0 or 1 to D, according to Z. D = {31'b0, Z}.		2	same	2	same	D		
396	WRNZ	D		Math and Logic	EEEE 1101011 000 DDDDDDDDD 001101111	.	Write 0 or 1 to D, according to !Z. D = {31'b0, !Z}.		2	same	2	same	D		
397	MODCZ	c,z	(WC/WZ/WCZ)	Math and Logic	EEEE 1101011 CZ1 0cccczzzz 001101111	.	Modify C and Z according to cccc and zzzz. C = cccc[{C,Z}], Z = zzzz[{C,Z}]. See "MODCZ Operand" list.		2	same	2	same			
398	MODC	c	(WC)	Math and Logic	EEEE 1101011 C01 0cccc0000 001101111	alias	Modify C according to cccc. C = cccc[{C,Z}]. See "MODCZ Operand" list.		2	same	2	same			
399	MODZ	z	(WZ)	Math and Logic	EEEE 1101011 Z01 00000zzzz 001101111	alias	Modify Z according to zzzz. Z = zzzz[{C,Z}]. See "MODCZ Operand" list.		2	same	2	same			
400	SETSCP	{#}D		Smart Pins	EEEE 1101011 00L DDDDDDDDD 001110000	.	Set four-channel oscilloscope enable to D[6] and set input pin base to D[5:2].		2	same	2	same			
401	GETSCP	D		Smart Pins	EEEE 1101011 000 DDDDDDDDD 001110001	.	Get four-channel oscilloscope samples into D. D = {ch3[7:0],ch2[7:0],ch1[7:0],ch0[7:0]}.		2	same	2	same			
402	JMP	#{\}A		Branch A - Jump	EEEE 1101100 RAA AAAAAAAAA AAAAAAAAA	.	Jump to A. If R = 1 then PC += A, else PC = A. "\" forces R = 0.		4	13...20	4	13...28			
403	CALL	#{\}A		Branch A - Call	EEEE 1101101 RAA AAAAAAAAA AAAAAAAAA	.	Call to A by pushing {C, Z, 10'b0, PC[19:0]} onto stack. If R = 1 then PC += A, else PC = A. "\" forces R = 0.		4	13...20	4	13...28			Push
404	CALLA	#{\}A		Branch A - Call	EEEE 1101110 RAA AAAAAAAAA AAAAAAAAA	.	Call to A by writing {C, Z, 10'b0, PC[19:0]} to hub long at PTRA++. If R = 1 then PC += A, else PC = A. "\" forces R = 0.		5...12 *	14...32 *	5...20 *	14...64 *		Write	
405	CALLB	#{\}A		Branch A - Call	EEEE 1101111 RAA AAAAAAAAA AAAAAAAAA	.	Call to A by writing {C, Z, 10'b0, PC[19:0]} to hub long at PTRB++. If R = 1 then PC += A, else PC = A. "\" forces R = 0.		5...12 *	14...32 *	5...20 *	14...64 *		Write	
406	CALLD	PA/PB/PTRA/PTRB,#{\}A		Branch A - Call	EEEE 11100WW RAA AAAAAAAAA AAAAAAAAA	.	Call to A by writing {C, Z, 10'b0, PC[19:0]} to PA/PB/PTRA/PTRB (per W). If R = 1 then PC += A, else PC = A. "\" forces R = 0.		4	13...20	4	13...28	Per W		
407	LOC	PA/PB/PTRA/PTRB,#{\}A		Math and Logic	EEEE 11101WW RAA AAAAAAAAA AAAAAAAAA	.	Get {12'b0, address[19:0]} into PA/PB/PTRA/PTRB (per W). If R = 1, address = PC + A, else address = A. "\" forces R = 0.		2	same	2	same	Per W		
408	AUGS	#n		Miscellaneous	EEEE 11110nn nnn nnnnnnnnn nnnnnnnnn	.	Queue #n to be used as upper 23 bits for next #S occurrence, so that the next 9-bit #S will be augmented to 32 bits.	✔	2	same	2	same			
409	AUGD	#n		Miscellaneous	EEEE 11111nn nnn nnnnnnnnn nnnnnnnnn	.	Queue #n to be used as upper 23 bits for next #D occurrence, so that the next 9-bit #D will be augmented to 32 bits.	✔	2		2				
410	RET_		<inst> <opa>	Instruction Prefix	0000 -------- --- --------- ---------	.	Execute <inst> and return if no branch. If <inst> is not branching then return by popping stack[19:0] into PC.		+2	+11...18	+2	+11...26			Pop
411	IF_NC_AND_NZ		<inst> <opa>	Instruction Prefix	0001 -------- --- --------- ---------	.	Execute <inst> if C = 0 and Z = 0.								
412	IF_NZ_AND_NC		<inst> <opa>	Instruction Prefix	0001 -------- --- --------- ---------	alias	Execute <inst> if C = 0 and Z = 0.								
413	IF_GT		<inst> <opa>	Instruction Prefix	0001 -------- --- --------- ---------	alias	Execute <inst> if C = 0 and Z = 0, or if 'greater than' after a comparison/subtraction.								
414	IF_A		<inst> <opa>	Instruction Prefix	0001 -------- --- --------- ---------	alias	Execute <inst> if C = 0 and Z = 0, or if 'above' after a comparison/subtraction.								
415	IF_00		<inst> <opa>	Instruction Prefix	0001 -------- --- --------- ---------	alias	Execute <inst> if C = 0 and Z = 0.								
416	IF_NC_AND_Z		<inst> <opa>	Instruction Prefix	0010 -------- --- --------- ---------	.	Execute <inst> if C = 0 and Z = 1.								
417	IF_Z_AND_NC		<inst> <opa>	Instruction Prefix	0010 -------- --- --------- ---------	alias	Execute <inst> if C = 0 and Z = 1.								
418	IF_01		<inst> <opa>	Instruction Prefix	0010 -------- --- --------- ---------	alias	Execute <inst> if C = 0 and Z = 1.								
419	IF_NC		<inst> <opa>	Instruction Prefix	0011 -------- --- --------- ---------	.	Execute <inst> if C = 0.								
420	IF_GE		<inst> <opa>	Instruction Prefix	0011 -------- --- --------- ---------	alias	Execute <inst> if C = 0, or if 'greater than or equal' after a comparison/subtraction.								
421	IF_AE		<inst> <opa>	Instruction Prefix	0011 -------- --- --------- ---------	alias	Execute <inst> if C = 0, or if 'above or equal' after a comparison/subtraction.								
422	IF_0X		<inst> <opa>	Instruction Prefix	0011 -------- --- --------- ---------	alias	Execute <inst> if C = 0.								
423	IF_C_AND_NZ		<inst> <opa>	Instruction Prefix	0100 -------- --- --------- ---------	.	Execute <inst> if C = 1 and Z = 0.								
424	IF_NZ_AND_C		<inst> <opa>	Instruction Prefix	0100 -------- --- --------- ---------	alias	Execute <inst> if C = 1 and Z = 0.								

order	#S = immediate (I=1). S = register. #D = immediate (L=1). D = register. - Assembly Syntax -	- Group -	- Encoding -	- Alias -	* Z = (result == 0). ** If #S and cogex, PC += signed(S). If #S and hubex, PC += signed(S*4). If S, PC = register S. - Description -	Next Inst Shielded from Interrupt	Clock Cycles (8 cogs) * +1 if crosses hub long - Cog Exec Mode - - LUT Exec Mode -	Clock Cycles (8 cogs) * +1 if crosses hub long - Hub Exec Mode -	Clock Cycles (16 cogs) * +1 if crosses hub long - Cog Exec Mode - - LUT Exec Mode -	Clock Cyles (16 cogs) * +1 if crosses hub long - Hub Exec Mode -	* Data not forwarded. - Register Write -	- Hub R/W -	- Stack R/W -
425	IF_10 <inst> <opa>	Instruction Prefix	0100 -------- --- --------- ---------	alias	Execute <inst> if C = 1 and Z = 0.							
426	IF_NZ <inst> <opa>	Instruction Prefix	0101 -------- --- --------- ---------	.	Execute <inst> if Z = 0.							
427	IF_NE <inst> <opa>	Instruction Prefix	0101 -------- --- --------- ---------	alias	Execute <inst> if Z = 0, or if 'not equal' after a comparison/subtraction.							
428	IF_X0 <inst> <opa>	Instruction Prefix	0101 -------- --- --------- ---------	alias	Execute <inst> if Z = 0.							
429	IF_C_NE_Z <inst> <opa>	Instruction Prefix	0110 -------- --- --------- ---------	.	Execute <inst> if C != Z.							
430	IF_Z_NE_C <inst> <opa>	Instruction Prefix	0110 -------- --- --------- ---------	alias	Execute <inst> if C != Z.							
431	IF_DIFF <inst> <opa>	Instruction Prefix	0110 -------- --- --------- ---------	alias	Execute <inst> if C != Z.							
432	IF_NC_OR_NZ <inst> <opa>	Instruction Prefix	0111 -------- --- --------- ---------	.	Execute <inst> if C = 0 or Z = 0.							
433	IF_NZ_OR_NC <inst> <opa>	Instruction Prefix	0111 -------- --- --------- ---------	alias	Execute <inst> if C = 0 or Z = 0.							
434	IF_NOT_11 <inst> <opa>	Instruction Prefix	0111 -------- --- --------- ---------	alias	Execute <inst> if C = 0 or Z = 0.							
435	IF_C_AND_Z <inst> <opa>	Instruction Prefix	1000 -------- --- --------- ---------	.	Execute <inst> if C = 1 and Z = 1.							
436	IF_Z_AND_C <inst> <opa>	Instruction Prefix	1000 -------- --- --------- ---------	alias	Execute <inst> if C = 1 and Z = 1.							
437	IF_11 <inst> <opa>	Instruction Prefix	1000 -------- --- --------- ---------	alias	Execute <inst> if C = 1 and Z = 1.							
438	IF_C_EQ_Z <inst> <opa>	Instruction Prefix	1001 -------- --- --------- ---------	.	Execute <inst> if C = Z.							
439	IF_Z_EQ_C <inst> <opa>	Instruction Prefix	1001 -------- --- --------- ---------	alias	Execute <inst> if C = Z.							
440	IF_SAME <inst> <opa>	Instruction Prefix	1001 -------- --- --------- ---------	alias	Execute <inst> if C = Z.							
441	IF_Z <inst> <opa>	Instruction Prefix	1010 -------- --- --------- ---------	.	Execute <inst> if Z = 1.							
442	IF_E <inst> <opa>	Instruction Prefix	1010 -------- --- --------- ---------	alias	Execute <inst> if Z = 1, or if 'equal' after a comparison/subtraction.							
443	IF_X1 <inst> <opa>	Instruction Prefix	1010 -------- --- --------- ---------	alias	Execute <inst> if Z = 1.							
444	IF_NC_OR_Z <inst> <opa>	Instruction Prefix	1011 -------- --- --------- ---------	.	Execute <inst> if C = 0 or Z = 1.							
445	IF_Z_OR_NC <inst> <opa>	Instruction Prefix	1011 -------- --- --------- ---------	alias	Execute <inst> if C = 0 or Z = 1.							
446	IF_NOT_10 <inst> <opa>	Instruction Prefix	1011 -------- --- --------- ---------	alias	Execute <inst> if C = 0 or Z = 1.							
447	IF_C <inst> <opa>	Instruction Prefix	1100 -------- --- --------- ---------	.	Execute <inst> if C = 1.							
448	IF_LT <inst> <opa>	Instruction Prefix	1100 -------- --- --------- ---------	alias	Execute <inst> if C = 1, or if 'less than' after a comparison/subtraction.							
449	IF_B <inst> <opa>	Instruction Prefix	1100 -------- --- --------- ---------	alias	Execute <inst> if C = 1, or if 'below' after a comparison/subtraction.							
450	IF_1X <inst> <opa>	Instruction Prefix	1100 -------- --- --------- ---------	alias	Execute <inst> if C = 1.							
451	IF_C_OR_NZ <inst> <opa>	Instruction Prefix	1101 -------- --- --------- ---------	.	Execute <inst> if C = 1 or Z = 0.							
452	IF_NZ_OR_C <inst> <opa>	Instruction Prefix	1101 -------- --- --------- ---------	alias	Execute <inst> if C = 1 or Z = 0.							
453	IF_NOT_01 <inst> <opa>	Instruction Prefix	1101 -------- --- --------- ---------	alias	Execute <inst> if C = 1 or Z = 0.							
454	IF_C_OR_Z <inst> <opa>	Instruction Prefix	1110 -------- --- --------- ---------	.	Execute <inst> if C = 1 or Z = 1.							
455	IF_Z_OR_C <inst> <opa>	Instruction Prefix	1110 -------- --- --------- ---------	alias	Execute <inst> if C = 1 or Z = 1.							
456	IF_LE <inst> <opa>	Instruction Prefix	1110 -------- --- --------- ---------	alias	Execute <inst> if C = 1 or Z = 1, or if 'less than or equal' after a comparison/subtraction.							
457	IF_BE <inst> <opa>	Instruction Prefix	1110 -------- --- --------- ---------	alias	Execute <inst> if C = 1 or Z = 1, or if 'below or equal' after a comparison/subtraction.							
458	IF_NOT_00 <inst> <opa>	Instruction Prefix	1110 -------- --- --------- ---------	alias	Execute <inst> if C = 1 or Z = 1.							
459	<inst> <opa>	Instruction Prefix	1111 -------- --- --------- ---------	.	Execute <inst> always. This is the default when no instruction prefix is expressed.							
460	_CLR	MODCZ Operand	cccc or zzzz = 0000	.	C/Z = 0							
461	_NC_AND_NZ	MODCZ Operand	cccc or zzzz = 0001	.	C/Z = !C AND !Z							
462	_NZ_AND_NC	MODCZ Operand	cccc or zzzz = 0001	alias	C/Z = !C AND !Z							
463	_GT	MODCZ Operand	cccc or zzzz = 0001	alias	C/Z = !C AND !Z, or 'greater than' after a comparison/subtraction.							
464	_NC_AND_Z	MODCZ Operand	cccc or zzzz = 0010	.	C/Z = !C AND Z							
465	_Z_AND_NC	MODCZ Operand	cccc or zzzz = 0010	alias	C/Z = !C AND Z							
466	_NC	MODCZ Operand	cccc or zzzz = 0011	.	C/Z = !C							
467	_GE	MODCZ Operand	cccc or zzzz = 0011	alias	C/Z = !C, or 'greater than or equal' after a comparison/subtraction.							
468	_C_AND_NZ	MODCZ Operand	cccc or zzzz = 0100	.	C/Z = C AND !Z							
469	_NZ_AND_C	MODCZ Operand	cccc or zzzz = 0100	alias	C/Z = C AND !Z							
470	_NZ	MODCZ Operand	cccc or zzzz = 0101	.	C/Z = !Z							
471	_NE	MODCZ Operand	cccc or zzzz = 0101	alias	C/Z = !Z, or 'not equal' after a comparison/subtraction.							
472	_C_NE_Z	MODCZ Operand	cccc or zzzz = 0110	.	C/Z = C NOT_EQUAL_TO Z							
473	_Z_NE_C	MODCZ Operand	cccc or zzzz = 0110	alias	C/Z = C NOT_EQUAL_TO Z							
474	_NC_OR_NZ	MODCZ Operand	cccc or zzzz = 0111	.	C/Z = !C OR !Z							
475	_NZ_OR_NC	MODCZ Operand	cccc or zzzz = 0111	alias	C/Z = !C OR !Z							
476	_C_AND_Z	MODCZ Operand	cccc or zzzz = 1000	.	C/Z = C AND Z							
477	_Z_AND_C	MODCZ Operand	cccc or zzzz = 1000	alias	C/Z = C AND Z							
478	_C_EQ_Z	MODCZ Operand	cccc or zzzz = 1001	.	C/Z = C EQUAL_TO Z							
479	_Z_EQ_C	MODCZ Operand	cccc or zzzz = 1001	alias	C/Z = C EQUAL_TO Z							
480	_Z	MODCZ Operand	cccc or zzzz = 1010	.	C/Z = Z							
481	_E	MODCZ Operand	cccc or zzzz = 1010	alias	C/Z = Z, or 'equal' after a comparison/subtraction.							
482	_NC_OR_Z	MODCZ Operand	cccc or zzzz = 1011	.	C/Z = !C OR Z							
483	_Z_OR_NC	MODCZ Operand	cccc or zzzz = 1011	alias	C/Z = !C OR Z							
484	_C	MODCZ Operand	cccc or zzzz = 1100	.	C/Z = C							
485	_LT	MODCZ Operand	cccc or zzzz = 1100	alias	C/Z = C, or 'less than' after a comparison/subtraction.							
486	_C_OR_NZ	MODCZ Operand	cccc or zzzz = 1101	.	C/Z = C OR !Z							
487	_NZ_OR_C	MODCZ Operand	cccc or zzzz = 1101	alias	C/Z = C OR !Z							
488	_C_OR_Z	MODCZ Operand	cccc or zzzz = 1110	.	C/Z = C OR Z							
489	_Z_OR_C	MODCZ Operand	cccc or zzzz = 1110	alias	C/Z = C OR Z							
490	_LE	MODCZ Operand	cccc or zzzz = 1110	alias	C/Z = C OR Z, or 'less than or equal' after a comparison/subtraction.							
491	_SET	MODCZ Operand	cccc or zzzz = 1111	.	C/Z = 1							